

# 北京理工大学

## 本科生毕业设计（论文）

### 基于微服务架构的在线视频分享平台的设计 与实现

Design and Implementation of an Online Video Sharing  
Platform Based on Microservices

|         |            |
|---------|------------|
| 学    院： | 计算机学院      |
| 专    业： | 计算机科学与技术   |
| 班    级： | 07812201   |
| 学生姓名：   | 孙将斌        |
| 学    号： | 1820221055 |
| 指导教师：   | 胡思康        |

2026 年 5 月 25 日

## 原创性声明

本人郑重声明：所呈交的毕业设计（论文），是本人在指导老师的指导下独立进行研究所取得的成果。除文中已经注明引用的内容外，本文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式标明。

特此申明。

本人签名：

日期：

年

月

日

## 关于使用授权的声明

本人完全了解北京理工大学有关保管、使用毕业设计（论文）的规定，其中包括：①学校有权保管、并向有关部门送交本毕业设计（论文）的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存本毕业设计（论文）；③学校可允许本毕业设计（论文）被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换本毕业设计（论文）；⑤学校可以公布本毕业设计（论文）的全部或部分内容。

本人签名：

日期：

年

月

日

指导老师签名：

日期：

年

月

日

# 基于微服务架构的在线视频分享平台的设计与实现

## 摘 要

随着移动互联网和在线视频业务的快速发展，视频分享平台已经成为用户获取信息、表达观点和参与社区互动的重要载体。传统单体架构在面对视频投稿、文件上传、视频转码、内容审核、搜索播放和互动统计等复杂业务时，容易出现模块耦合度高、扩展困难和维护成本较高等问题。针对上述问题，本文设计并实现了一个基于微服务架构的在线视频分享平台。

系统采用前后端分离与微服务相结合的设计思路，后端基于 Spring Boot 和 Spring Cloud Alibaba 构建，按照业务职责划分为网关服务、主站业务服务、资源文件服务、互动服务、后台管理服务和公共基础模块。系统使用 MySQL 保存核心业务数据，Redis 处理验证码、登录态、缓存和临时任务状态，Elasticsearch 支持视频全文检索，RabbitMQ 承担 AI 审核任务的异步消息通信，Seata 用于支撑关键跨服务写操作的一致性控制。在视频处理方面，系统实现了视频分片上传、文件合并、FFmpeg 转码和 HLS 播放资源生成；在内容审核方面，系统设计了独立的 AI 审核服务，对视频语音、声音事件和关键帧内容进行分析，并将风险等级、风险标签和审核建议返回后台，由管理员进行人工终审。

测试结果表明，系统能够完成视频浏览、搜索播放、用户互动、视频投稿、分片上传、视频转码、AI 初审、人工审核和正式发布等核心流程，整体业务链路能够形成闭环。本文的研究与实现验证了微服务架构、异步任务处理、视频资源处理和智能审核技术在在线视频分享平台中的综合应用价值。

**关键词：**在线视频分享平台；微服务架构；分片上传；视频转码；智能内容审核

## **Design and Implementation of an Online Video Sharing Platform Based on Microservices**

### **Abstract**

With the rapid development of mobile Internet and online video services, video sharing platforms have become important carriers for users to obtain information, express opinions, and participate in community interactions. However, when traditional monolithic architecture is applied to complex business processes such as video submission, file uploading, video transcoding, content review, search, playback, and interaction statistics, it may lead to high module coupling, poor scalability, and increased maintenance cost. To address these problems, this thesis designs and implements an online video sharing platform based on microservices.

The system adopts a combination of front-end and back-end separation and microservice architecture. The back-end is built with Spring Boot and Spring Cloud Alibaba, and is divided into several modules according to business responsibilities, including the gateway service, main business service, resource service, interaction service, administration service, and common basic modules. MySQL is used to store core business data, Redis is used for verification codes, login states, cache data, and temporary task states, Elasticsearch is used for full-text video search, RabbitMQ is used for asynchronous communication of AI review tasks, and Seata is introduced to support data consistency in key cross-service write operations. For video processing, the system implements chunked uploading, file merging, FFmpeg transcoding, and HLS playback resource generation. For content review, an independent AI review service is designed to analyze video speech, sound events, and key frames, and then return risk levels, risk labels, and review suggestions to the administration system for final manual review.

The test results show that the system can complete core processes such as video browsing, search and playback, user interaction, video submission, chunked uploading, video transcoding, AI preliminary review, manual review, and final publishing. The overall business process forms a complete closed loop. The design and implementation of this thesis verify the comprehensive application value of microservice architecture, asynchronous task



processing, video resource processing, and intelligent content review technology in an on-line video sharing platform.

**Key Words: Online Video Sharing Platform; Microservice Architecture; Chunked Uploading; Video Transcoding; Intelligent Content Review**

## 目 录

|                                |    |
|--------------------------------|----|
| 摘 要 .....                      | I  |
| Abstract .....                 | II |
| 第 1 章 绪论 .....                 | 1  |
| 1.1 研究背景与意义 .....              | 1  |
| 1.2 国内外研究现状 .....              | 1  |
| 1.3 研究内容与目标 .....              | 2  |
| 1.4 论文结构安排 .....               | 3  |
| 第 2 章 相关技术介绍 .....             | 5  |
| 2.1 微服务架构与服务治理技术 .....         | 5  |
| 2.1.1 微服务架构思想 .....            | 5  |
| 2.1.2 服务构建与运行基础 .....          | 6  |
| 2.1.3 服务注册、网关与远程调用 .....       | 6  |
| 2.2 数据存储、缓存与检索技术 .....         | 7  |
| 2.2.1 关系型数据库与数据访问层 .....       | 7  |
| 2.2.2 Redis 缓存与临时状态管理 .....    | 7  |
| 2.2.3 Elasticsearch 全文检索 ..... | 8  |
| 2.3 视频上传、转码与播放技术 .....         | 8  |
| 2.3.1 大文件分片上传 .....            | 8  |
| 2.3.2 FFmpeg 媒体转码 .....        | 8  |
| 2.3.3 HLS 流媒体播放 .....          | 9  |
| 2.3.4 浏览器视频播放能力 .....          | 9  |
| 2.4 前端页面开发与接口调用技术 .....        | 10 |
| 2.5 异步通信与任务处理技术 .....          | 10 |
| 2.5.1 异步处理机制 .....             | 10 |
| 2.5.2 RabbitMQ 消息队列 .....      | 10 |
| 2.5.3 任务状态与一致性控制 .....         | 11 |
| 2.6 智能内容审核技术 .....             | 11 |
| 2.6.1 视频内容审核思路 .....           | 11 |
| 2.6.2 语音识别与声音事件识别 .....        | 11 |
| 2.6.3 多模态理解与人机协同审核 .....       | 12 |

|                            |    |
|----------------------------|----|
| 第 3 章 系统需求分析 .....         | 13 |
| 3.1 系统总体需求分析 .....         | 13 |
| 3.1.1 需求分析目标 .....         | 13 |
| 3.1.2 系统参与角色划分 .....       | 14 |
| 3.1.3 系统总体业务流程 .....       | 15 |
| 3.2 用户端功能需求分析 .....        | 16 |
| 3.2.1 游客访问与用户认证需求 .....    | 17 |
| 3.2.2 视频浏览、搜索与播放需求 .....   | 17 |
| 3.2.3 社区互动需求 .....         | 17 |
| 3.3 创作中心功能需求分析 .....       | 18 |
| 3.3.1 视频投稿需求 .....         | 18 |
| 3.3.2 视频上传与文件处理需求 .....    | 18 |
| 3.3.3 稿件状态与作品管理需求 .....    | 19 |
| 3.4 管理端与 AI 审核需求分析 .....   | 19 |
| 3.4.1 管理员认证与分类管理需求 .....   | 19 |
| 3.4.2 人工审核与 AI 审核需求 .....  | 20 |
| 3.4.3 视频状态流转需求 .....       | 20 |
| 3.5 非功能需求分析 .....          | 21 |
| 第 4 章 系统架构及数据库设计 .....     | 22 |
| 4.1 系统总体架构设计 .....         | 22 |
| 4.1.1 系统总体架构 .....         | 22 |
| 4.2 微服务模块划分 .....          | 23 |
| 4.2.1 系统服务模块说明 .....       | 23 |
| 4.2.2 服务协作关系 .....         | 24 |
| 4.3 数据库设计 .....            | 25 |
| 4.3.1 核心数据表与投稿发布数据设计 ..... | 25 |
| 4.3.2 AI 审核数据设计 .....      | 26 |
| 4.4 消息通信与 AI 审核任务设计 .....  | 27 |
| 4.4.1 消息通信设计 .....         | 28 |
| 4.4.2 AI 审核消息通道设计 .....    | 29 |
| 4.5 视频业务流程设计 .....         | 29 |
| 4.5.1 视频投稿与资源处理流程设计 .....  | 29 |
| 4.5.2 视频审核与发布流程设计 .....    | 30 |

|                               |    |
|-------------------------------|----|
| 4.5.3 视频消费与互动流程设计 .....       | 31 |
| 第 5 章 视频分享平台的设计与实现 .....      | 32 |
| 5.1 用户端功能模块实现 .....           | 32 |
| 5.1.1 用户端整体功能实现 .....         | 32 |
| 5.1.2 首页浏览与视频搜索实现 .....       | 33 |
| 5.1.3 视频播放页实现 .....           | 35 |
| 5.2 创作中心与视频投稿实现 .....         | 37 |
| 5.2.1 创作中心功能结构 .....          | 37 |
| 5.2.2 视频分片上传实现 .....          | 38 |
| 5.2.3 投稿信息保存与状态初始化实现 .....    | 40 |
| 5.3 视频转码与资源服务实现 .....         | 41 |
| 5.3.1 资源服务职责 .....            | 41 |
| 5.3.2 视频文件合并与转码实现 .....       | 42 |
| 5.3.3 HLS 播放资源生成与访问实现 .....   | 44 |
| 5.4 互动功能模块实现 .....            | 45 |
| 5.4.1 评论与弹幕实现 .....           | 46 |
| 5.4.2 点赞、收藏、投币与关注实现 .....     | 49 |
| 5.4.3 互动统计数据更新实现 .....        | 49 |
| 5.5 后台管理功能实现 .....            | 50 |
| 5.5.1 后台管理端整体实现 .....         | 50 |
| 5.5.2 分类与内容管理实现 .....         | 52 |
| 5.5.3 视频审核管理实现 .....          | 52 |
| 5.6 AI 审核任务编排与人机协同实现 .....    | 53 |
| 5.6.1 AI 审核任务创建与消息投递实现 .....  | 55 |
| 5.6.2 审核结果接收与任务状态更新实现 .....   | 56 |
| 5.6.3 AI 初审与人工终审协同实现 .....    | 57 |
| 5.6.4 审核通过与发布数据同步实现 .....     | 57 |
| 5.7 多模态 AI 审核服务实现 .....       | 58 |
| 5.7.1 AI 审核服务结构与模型选型依据 .....  | 58 |
| 5.7.2 视频预处理与音频提取实现 .....      | 60 |
| 5.7.3 语音文本、声音事件与关键帧审核实现 ..... | 61 |
| 5.7.4 综合风险结果生成实现 .....        | 65 |

|                        |    |
|------------------------|----|
| 第 6 章 系统测试与结果分析 .....  | 67 |
| 6.1 测试环境与测试方案 .....    | 67 |
| 6.1.1 测试环境配置 .....     | 67 |
| 6.1.2 测试方案设计 .....     | 68 |
| 6.2 系统功能测试 .....       | 69 |
| 6.3 视频处理与发布链路测试 .....  | 70 |
| 6.4 AI 审核与异常场景测试 ..... | 71 |
| 6.5 跨服务事务一致性测试 .....   | 72 |
| 6.6 测试结果分析 .....       | 73 |
| 6.6.1 测试结果汇总 .....     | 73 |
| 结    论 .....           | 74 |
| 参考文献 .....             | 75 |
| 致    谢 .....           | 77 |

## 第1章 绪论

### 1.1 研究背景与意义

随着移动互联网、智能终端以及网络带宽的持续发展，在线视频已经成为了互联网内容传播的一种重要形式。用户不再仅仅是被动观看视频，同时还会凭借投稿、评论、弹幕、点赞、收藏、投币和关注等多种方式，参与到内容生产与社区互动当中。因此，一个完整的视频分享平台不仅要提供视频播放方面的支持，还需要拥有内容生产、资源处理、用户互动、内容检索和后台治理等方面的能力。

在线视频平台的业务链路相对较长，其中会涉及到用户管理、视频投稿、文件上传、视频转码、内容审核、搜索播放以及互动统计等多个环节。若所有功能集中在单体系统中，随着功能不断增加，容易出现模块间耦合程度高、维护工作开展困难以及扩展不便等问题。微服务架构强调围绕业务能力拆分服务，各服务可独立开发、部署和扩展，因此适用于功能模块较多、业务边界较为清晰的 Web 平台系统<sup>[1-3]</sup>。

与此同时，用户生成内容的数量在不断地增加，平台所面临的内容安全问题也随之变得更加突出。仅仅依赖人工审核会带来审核效率低下、工作量庞大以及响应不够及时等方面的问题。语音识别、声音事件检测以及多模态模型的持续发展，为视频内容审核工作提供了新的技术手段。将 AI 初审结果提供给管理员作为参考，在保留人工终审的基础上提高审核效率，形成“AI 初审 + 人工终审”的协同审核流程<sup>[4-6]</sup>。

借由上述背景，本文设计并实现了一个建立在微服务架构之上的在线视频分享平台。系统功能围绕普通用户、内容创作者以及管理员这三类主要对象展开，完成视频浏览、搜索、投稿、转码、互动、后台审核以及 AI 辅助审核等功能。本课题的实践意义主要体现在：一方面，借助微服务拆分方式，有助于提升系统结构的清晰程度与可维护性；另一方面，凭借视频处理、异步任务和智能审核等模块，构建一个相对完整的视频平台业务闭环。

### 1.2 国内外研究现状

国外在线视频平台起步的时间比较早，像 YouTube、Netflix 等平台在视频分发、流媒体播放、内容推荐以及系统架构这些方面，都积累了比较多的实践经验。随着云计算和微服务技术的持续发展，大型内容平台普遍会借助服务拆分、负载均衡、弹性

扩展和自动化部署等手段支撑复杂业务。与此同时，HLS 等流媒体协议以及 FFmpeg 等开源音视频工具，也被广泛地运用在了视频转码、切片和格式转换等场景当中<sup>[7-8]</sup>。

国内在线视频和短视频平台的发展速度很快，像哔哩哔哩、抖音、快手等平台，在内容创作、弹幕互动、社区运营以及内容审核等方面，都已经形成了比较成熟的业务模式。国内企业在后端的架构上，也大量地选用了 Spring Cloud、Nacos、Redis、Elasticsearch、RabbitMQ 等一系列技术，从而应对业务模块复杂、数据读写频繁以及服务扩展困难等方面的挑战<sup>[9-13]</sup>。

在内容审核这一方面，国内外的平台都已经逐步地从单纯的人工审核，转向了“机器审核 + 人工复核”这样一种模式。传统的审核方式主要依靠关键词、规则以及简单的分类模型，当面对复杂语义和音视频混合在一起的内容时，会表现出一定的局限性。近年来，多模态人工智能技术已可对语音文本、声音事件和画面内容进行综合性分析，这为视频审核工作提供了更加丰富的判断依据<sup>[4,6,14]</sup>。

从总体上来看，在线视频平台所涉及的相关技术已经发展得比较成熟，不过，对于毕业设计这个项目而言，其难点在于如何把微服务架构、视频处理、用户互动、搜索功能以及 AI 审核服务整合成一个可实际运行的系统。本文在现有技术基础之上，构建一个具备核心业务闭环的视频分享平台原型，借此体现相关技术在复杂 Web 系统中的综合应用价值。

### 1.3 研究内容与目标

本文围绕视频平台中的用户管理、视频投稿、资源处理、互动行为、后台审核以及 AI 内容审核等各项功能，采用前后端分离架构，后端以 Spring Cloud Alibaba 微服务体系为基础，AI 审核采用独立的 Python 服务实现。

主要研究内容如下：

1. 完成在线视频分享平台的需求分析与总体设计的相关工作。根据平台业务的特性，明确普通用户、内容创作者、管理员以及 AI 审核服务各自的功能边界，并且对系统的总体架构和服务模块划分进行设计。
2. 实现视频投稿、上传、转码以及播放这一整条链路。系统会提供对创作者上传视频和封面的支持，把投稿信息保存下来，并且借助资源服务来完成视频的合并、转码以及 HLS 播放资源的生成工作<sup>[7-8]</sup>。

3. 实现视频浏览、搜索以及互动方面的功能。用户浏览公开视频、按照关键词对视频进行检索、进入视频详情页播放视频，并且进行评论、弹幕、点赞、收藏、投币和关注等各类互动操作。
4. 实现后台管理以及人工审核的相关功能。管理员可维护视频分类、查看待审核稿件和 AI 审核结果，并且对视频作出通过或驳回处理。
5. 实现一个独立的 AI 多模态审核服务。AI 服务通过消息队列来接收审核任务，对视频中的语音、声音事件以及关键帧内容进行分析，并且把风险等级、风险标签还有审核建议返回给后端[4,6,13,15]。

为此完成一个功能相对完整、结构清晰且流程形成闭环的在线视频分享平台原型。该原型体现了微服务架构在复杂业务拆分当中的具体应用，覆盖视频从上传、处理、审核再到发布播放的主要流程，并且借助 AI 审核服务，提高内容审核的辅助决策能力。

## 1.4 论文结构安排

本文一共划分成了七个章节，第一章为绪论，主要介绍课题的研究背景与意义、国内外研究现状、本文的研究内容与目标，并对论文整体结构进行说明。

第二章为相关技术介绍，主要介绍系统开发过程中使用到的微服务架构、数据存储、缓存检索、视频上传转码、异步通信以及智能内容审核等相关技术。

第三章为系统需求分析，主要从系统总体需求、参与角色划分、用户端功能需求、创作中心功能需求、管理端与 AI 审核需求以及非功能需求等方面，对系统需要实现的功能和质量要求进行分析。

第四章为系统架构及数据库设计，主要围绕系统总体架构、功能模块划分、业务流程设计、数据库设计、接口与消息通信设计以及 AI 审核任务设计等内容展开，为后续系统实现提供设计依据。

第五章为视频分享平台的设计与实现，主要说明用户端核心功能、创作中心与视频投稿、视频资源处理、互动功能、后台管理功能以及 AI 审核任务编排与人机协同等核心模块的具体实现过程。

第六章为系统测试与结果分析，主要对系统测试环境、系统功能、视频处理与发布链路、AI 审核与异常场景等内容进行测试，并对测试结果进行总结评价。



最后为结论，主要对本文完成的主要工作进行总结，归纳系统的技术特点与创新点，分析当前系统存在的不足，并提出后续可行的改进方向。

## 第2章 相关技术介绍

在线视频分享平台的实现需要多个技术环节协同完成。系统既要处理用户、视频、评论、审核等结构化业务数据，也要支持大文件上传、视频转码、流媒体播放、全文检索、异步任务和智能内容审核等能力。因此，本章围绕论文设计与实现中的关键技术进行说明。

### 2.1 微服务架构与服务治理技术

#### 2.1.1 微服务架构思想

在功能较少的系统中，单体架构可降低开发和部署成本。但在线视频分享平台涉及用户认证、视频投稿、资源转码、搜索检索、互动评论、后台审核和内容安全等多个业务领域。如果所有功能都集中在一个应用中，代码耦合度会逐渐升高，后续扩展和维护也会变得困难。

微服务架构强调的是围绕业务能力来拆分服务，每个服务负责相对独立的职责，并且借助轻量级通信方式来进行协作<sup>[1]</sup>。对于视频平台而言，视频资源处理、普通业务管理以及智能审核任务在数据形态、处理耗时和访问压力这些方面存在明显差异，采用服务化拆分有助于降低模块之间的耦合程度。

服务拆分并不是简单按照代码目录来划分，而是要结合业务职责、数据边界以及访问压力进行分析。表 2-1 把服务拆分时需要关注的因素做了概括。

表 2-1 服务拆分原则概括

| 原则     | 说明                             |
|--------|--------------------------------|
| 职责清晰   | 每个服务围绕明确业务能力设计，避免一个服务承担过多无关功能  |
| 高内聚低耦合 | 服务内部功能联系紧密，服务之间减少不必要的直接依赖      |
| 数据边界明确 | 尽量让核心数据归属于对应服务，降低跨服务频繁读写带来的复杂度 |
| 便于扩展   | 对访问压力大或处理耗时的模块，可单独扩展和部署        |
| 运维可控   | 服务数量需要与项目规模匹配，避免过度拆分导致管理成本过高   |

### 2.1.2 服务构建与运行基础

Spring Boot 是 Java Web 开发当中比较常用的一种服务构建框架。它借助自动配置、起步依赖以及内嵌服务器这些机制来减少重复配置，使开发者可较快搭建独立运行的后端服务<sup>[16]</sup>。在这类系统当中，用户服务、资源服务、互动服务以及后台管理服务基于 Spring Boot 构建。

当系统拆分出多个服务之后，还需处理服务注册、配置管理、网关路由以及远程调用这些问题。Spring Cloud 提供了一套微服务开发组件，用来支持分布式系统当中的服务治理能力<sup>[9]</sup>。因此，Spring Boot 更侧重于单个服务的快速构建，而 Spring Cloud 则更侧重于多个服务之间的协同管理。

### 2.1.3 服务注册、网关与远程调用

在微服务环境当中，服务实例可能会因为重启、扩容或者部署迁移而改变地址。若调用方直接固定服务地址，会降低系统的灵活性和可维护性。服务注册与发现机制能让服务启动之后向注册中心登记自身信息，调用方再借助服务名获取可用实例。Nacos 支持服务发现和配置管理，比较适合用在微服务系统里的注册中心和配置中心<sup>[10]</sup>。

系统对外访问通常还需要一个统一的入口。Spring Cloud Gateway 是 Spring Cloud 体系当中的网关组件，它依据路由规则把请求转发到不同的服务，并且在网关层统一处理跨域、鉴权、日志以及内部接口隔离这些通用逻辑<sup>[17]</sup>。引入网关减少客户端直接感知多个内部服务所带来的复杂度。

服务之间的同步调用借助 OpenFeign 来实现。OpenFeign 是一个声明式的 HTTP 客户端，开发者通过接口描述远程调用关系，由框架完成请求的发送以及响应的转换<sup>[18]</sup>。不过同步调用比较适合那些耗时较短、需要马上返回结果的场景，而视频转码和 AI 审核这类长耗时任务则更适合使用异步消息机制。

在跨服务写操作的场景当中，单个服务的本地事务很难覆盖完整的业务链路。Seata 是一个面向微服务架构的分布式事务解决方案，它能通过全局事务协调机制处理跨服务的数据提交与回滚问题<sup>[19]</sup>。本系统在公共基础模块当中引入了 Seata，并在涉及投稿审核、互动计数以及用户资产变更这类关键写操作时，为数据一致性控制提供技术方面的支撑。

表 2-2 把微服务治理相关的技术做了一个概括总结。

表 2-2 微服务治理相关技术作用

| 技术                   | 主要作用              | 适用环节           |
|----------------------|-------------------|----------------|
| Spring Boot          | 快速构建独立运行的后端服务     | 各业务服务基础框架      |
| Spring Cloud         | 提供微服务治理组件         | 多服务协同与治理       |
| Nacos                | 服务注册发现、配置管理       | 服务地址管理与配置集中维护  |
| Spring Cloud Gateway | 统一系统入口与请求转发       | 前端访问、鉴权、内部接口保护 |
| OpenFeign            | 声明式远程调用           | 服务之间的短耗时同步调用   |
| Seata                | 分布式事务协调、全局提交与回滚控制 | 跨服务写操作的数据一致性控制 |

## 2.2 数据存储、缓存与检索技术

### 2.2.1 关系型数据库与数据访问层

关系型数据库比较适合保存结构清晰、关系明确的业务数据。MySQL 是一种常用的关系型数据库管理系统，它拥有成熟的事务、索引以及查询能力<sup>[20]</sup>。在在线视频分享平台当中，用户信息、视频基本信息、分类信息、评论记录以及审核记录这些数据都属于典型的结构化业务数据，适合由关系型数据库来统一管理。

为了避免在业务代码当中直接拼接大量 SQL，Java 后端通常会引入数据持久层框架。MyBatis 支持通过 XML 或者注解来编写 SQL，并且把查询结果映射为 Java 对象<sup>[21]</sup>。跟完全自动化的 ORM 框架相比，MyBatis 保留了较强的 SQL 控制能力，比较适合查询条件比较灵活的业务系统。

### 2.2.2 Redis 缓存与临时状态管理

缓存技术主要用来提升热点数据的访问速度，同时减少数据库的压力。Redis 是一种基于内存的数据存储系统，它支持字符串、列表、集合、有序集合以及哈希这些数据结构<sup>[11]</sup>。由于其读写速度比较快、数据结构也比较灵活，所以经常被用在验证码、登录状态、访问计数、分类缓存以及临时任务状态等场景当中。

需要留意的是，Redis 更适合用来保存热点数据和临时状态，不能简单地替代关系型数据库。核心业务数据仍保存在 MySQL 当中，而缓存层则用来提升访问效率或者辅助异步流程。让正式数据和临时数据各自承担起适宜的职责。

### 2.2.3 Elasticsearch 全文检索

视频平台的搜索功能不仅需要根据标题和简介进行关键词匹配，还要考虑分词、相关性排序以及搜索结果召回这些问题。普通数据库虽然也能做模糊查询，但在文本内容比较多、检索条件比较复杂的时候，效率和效果都会受到限制。

Elasticsearch 是一个分布式的搜索与分析引擎，它基于倒排索引来实现全文检索，经常被用在搜索、日志分析以及复杂查询这些场景当中<sup>[12]</sup>。在内容类的平台里，Elasticsearch 通常作为业务数据库之外的检索索引层来使用。MySQL 负责保存权威的业务数据，Elasticsearch 则负责提升搜索体验。

表 2-3 数据处理相关技术对比

| 技术            | 主要特点          | 适用场景               |
|---------------|---------------|--------------------|
| MySQL         | 结构化存储、事务能力成熟  | 用户、视频、评论、审核等核心业务数据 |
| MyBatis       | SQL 可控、便于对象映射 | 数据访问层封装和复杂查询组织     |
| Redis         | 内存读写快、数据结构灵活  | 验证码、登录态、缓存、计数和临时状态 |
| Elasticsearch | 倒排索引、全文检索能力强  | 视频标题、简介、标签等内容检索    |

## 2.3 视频上传、转码与播放技术

### 2.3.1 大文件分片上传

视频文件通常体积比较大，若一次性上传完整文件，很容易受到网络波动、接口超时以及浏览器限制的影响。分片上传的基本思路是把一个大文件划分成多个较小的片段，由客户端逐个进行上传，服务端记录上传进度，等所有分片都到达之后再完成合并。

分片上传可降低单次请求失败所带来的重传成本，也便于系统记录上传状态。对于视频投稿这个场景来说，分片上传有助于提升大文件上传的稳定性，也是后续转码和审核流程可顺利执行的基础。

### 2.3.2 FFmpeg 媒体转码

用户上传的视频可能来自不同的设备和软件，视频格式、编码方式、分辨率以及码率并不统一。若系统直接提供原始文件进行播放，可能出现浏览器兼容性差、加

载慢或者播放失败等问题。因此，视频平台通常需要对上传的文件进行统一的转码。

FFmpeg 是一个常用的开源音视频处理工具，其支持视频转码、格式转换、音频提取、截图以及切片等操作<sup>[7]</sup>。借助转码把不同来源的视频转换成适合在线播放的格式，并且按需生成封面、音频分析文件或者流媒体播放资源。

媒体转码属于比较耗时的操作，不适合放在用户上传请求当中同步完成。更合理的方式是，在用户提交视频之后生成后台任务，由异步消费者完成转码处理，然后再把处理状态回写到数据库或者缓存当中。

### 2.3.3 HLS 流媒体播放

在线播放视频的时候，若直接加载完整视频文件，会增加初始等待时间，也不利于弱网环境下的播放体验。HLS 是一种基于 HTTP 的流媒体传输协议，它把视频组织成播放列表和多个媒体分片，播放器先读取播放列表，再按需加载对应片段<sup>[8]</sup>。

HLS 常见的文件包括 .m3u8 播放列表和 .ts 媒体分片。用户看到的是连续播放的视频，而系统实际提供的是一组按需加载的片段资源。这种方式有利于降低视频初始加载的压力，也便于后续扩展多清晰度的播放。

### 2.3.4 浏览器视频播放能力

现代浏览器借助 HTML5 的 video 元素来提供视频播放能力，支持播放、暂停、音量控制、进度控制以及全屏这些基本操作<sup>[22]</sup>。不过，完整的视频播放体验不仅依赖浏览器的控件，还需要服务端提供兼容的视频格式、稳定的资源路径以及合理的加载策略。

视频播放链路能概括为“上传文件、进行转码处理、生成播放资源、由浏览器加载并播放”。各个技术环节的作用如表 2-4 所示。

表 2-4 视频处理相关技术作用

| 技术环节        | 解决的问题          | 作用          |
|-------------|----------------|-------------|
| 分片上传        | 大文件上传失败后重传成本高  | 提升上传稳定性     |
| FFmpeg 转码   | 原始格式不统一、播放兼容性差 | 生成统一播放资源    |
| HLS 分片      | 完整加载视频等待时间长    | 支持按需加载和在线播放 |
| HTML5 video | 浏览器页面播放控制      | 提供前端播放能力    |

## 2.4 前端页面开发与接口调用技术

系统前端包含普通用户端和后台管理端两个部分，页面开发采用 Vue 3 与 Vite 作为基础工具链。Vue 3 负责组件化页面构建，Vite 用于提升开发启动和构建效率；Vue Router 用于管理首页、播放页、创作中心、个人中心和后台管理页面之间的路由跳转；Pinia 用于维护用户登录态、页面共享状态和部分业务状态<sup>[23-26]</sup>。

后台管理端需要较多表格、表单、弹窗、分页和审核操作组件，因此采用 Element Plus 提供基础 UI 组件；前端与后端服务之间通过 Axios 发起 HTTP 请求，并由网关统一转发到对应微服务<sup>[27-28]</sup>。这种前后端分离方式可降低页面展示逻辑与后端业务逻辑之间的耦合，也便于普通用户端和管理端分别维护。

## 2.5 异步通信与任务处理技术

### 2.5.1 异步处理机制

在 Web 系统当中，并不是所有的操作都适合马上完成并返回结果。对于视频转码、文件清理、AI 审核这类比较耗时的任务，若让前台请求一直等待，很容易造成接口超时，也会降低用户体验。数据密集型应用设计当中强调，可靠性与可维护性需要结合任务状态、故障恢复以及系统解耦来加以考虑<sup>[29]</sup>。

异步处理的基本思想是：用户先提交任务，系统记录任务状态，然后由后台消费者执行具体任务。用户或管理员可通过状态查看处理进度。这样既能减少前台请求等待时间，也能提高复杂任务处理的稳定性。

### 2.5.2 RabbitMQ 消息队列

RabbitMQ 是一种消息队列中间件，支持交换机、队列、路由键、消息确认、持久化和死信队列等机制<sup>[13]</sup>。在视频平台中，RabbitMQ 负责连接“任务生产者”和“任务消费者”，例如上传服务产生转码任务，资源服务或转码消费者异步处理视频文件。

消息队列可降低服务之间的直接耦合程度，但同时也会带来消息重复、消费失败、任务幂等以及状态补偿这些问题。因此，系统在使用消息队列的时候，不能只是发送消息，还需要配合任务状态表、失败重试以及异常记录这些机制。

### 2.5.3 任务状态与一致性控制

异步任务跟同步请求的主要区别在于，异步任务通常要经历待处理、处理中、处理成功、处理失败等多个状态。微服务模式当中也强调，事件驱动的流程往往需要配合状态记录、重试以及补偿机制来使用<sup>[3]</sup>。

在视频平台当中，转码任务、审核任务以及文件清理任务都需要有明确的状态流转。任务状态既可在前端展示处理进度，也可在后台排查异常以及避免重复处理。对于那些不要求强一致性、不要求立即完成的业务，可采用最终一致和异步补偿的策略，以此减少强事务带来的复杂度。

表 2-5 同步调用与异步通信对比

| 比较项  | 同步调用          | 异步通信            |
|------|---------------|-----------------|
| 适用任务 | 查询、登录、短时间业务处理 | 转码、审核、文件处理等耗时任务 |
| 用户体验 | 需要等待处理结果      | 前台快速返回，后台继续执行   |
| 服务耦合 | 调用方依赖被调用方即时响应 | 生产者和消费者相对解耦     |
| 异常处理 | 失败可立即返回       | 需要记录状态、重试和补偿    |

## 2.6 智能内容审核技术

### 2.6.1 视频内容审核思路

内容审核的目标是在内容正式发布之前识别出可能存在的违规或者风险信息。传统审核主要依赖人工判断或者规则匹配，而算法内容审核通常需要跟人工判断、规则体系以及申诉机制结合起来使用<sup>[30]</sup>。

视频内容比普通的文本和图片要更复杂。它同时包含了画面、语音、背景声音以及上下文语义，某些风险信息可能隐藏在语音内容或者关键画面当中。智能内容审核的作用就是借助语音识别、声音事件识别以及多模态理解这些技术，为人工审核提供辅助依据，而不是完全替代人工判断。

### 2.6.2 语音识别与声音事件识别

视频当中的语音内容往往带有比较重要的语义信息。Whisper 是一种基于大规模弱监督训练的语音识别模型，它在多语言语音识别任务上有着较好的表现<sup>[4]</sup>。借助语



音识别，把视频当中的人声转换成文本，再交由规则或者模型做进一步分析。

除了语音内容之外，声音事件也可能提示视频存在风险。比如说尖叫、爆炸声、警报声这类异常声音，可能对应着特殊的场景。AudioSet 是一个大规模音频事件数据集，它为声音事件识别方面的研究提供了基础<sup>[5]</sup>。YAMNet 这类声音事件分类模型关注的是“发生了什么声音”，这与语音识别关注“说了什么内容”有所不同。

### 2.6.3 多模态理解与人机协同审核

多模态语义理解指的是模型同时处理图像、文本、语音等多种信息，并且综合判断内容的含义。Qwen-VL 是一个视觉语言模型，它可结合图像和文本进行理解，用于图像描述、视觉问答、文字识别以及多模态推理等任务<sup>[6]</sup>。在视频审核场景，从视频里抽取出关键帧，再结合语音转写出来的文本进行综合分析。

考虑到智能模型可能存在误判、漏判或者推理结果不稳定这些问题，内容审核系统更适合采用“AI 做初审，人工做终审”的方式。AI 审核负责快速发现风险片段、生成风险等级以及审核建议；管理员再根据模型结果、视频内容以及平台规则来做出最终的决策。这种方式可提高审核效率，也可保留人工判断的灵活性。

表 2-6 智能审核相关技术作用

| 技术      | 分析对象        | 审核作用               |
|---------|-------------|--------------------|
| Whisper | 视频语音内容      | 将语音转换为文本，辅助识别风险表达  |
| YAMNet  | 背景声音和事件声音   | 发现异常声音事件，提供辅助线索    |
| Qwen-VL | 关键帧与文本信息    | 综合画面和语义，生成风险判断依据   |
| 人工审核    | 视频整体内容与平台规则 | 对 AI 结果进行复核并作出最终决策 |

第 3 章 系统需求分析

本章主要围绕系统总体需求、用户端需求、创作中心需求、管理端与 AI 审核需求、非功能需求以及可行性分析来展开。

3.1 系统总体需求分析

3.1.1 需求分析目标

系统围绕视频内容的生产、处理、审核、发布以及消费形成一个完整的闭环。普通用户可浏览、搜索和播放视频，并参与评论、弹幕、点赞、收藏、投币这类互动；内容创作者进行视频的上传、编辑投稿以及个人作品管理；管理员在后台开展分类维护、视频审核以及内容治理方面的工作；AI 审核服务则负责对上传的视频进行自动化风险识别，以此为人工审核提供辅助依据。图 3-1 展示了这个系统的功能结构图。

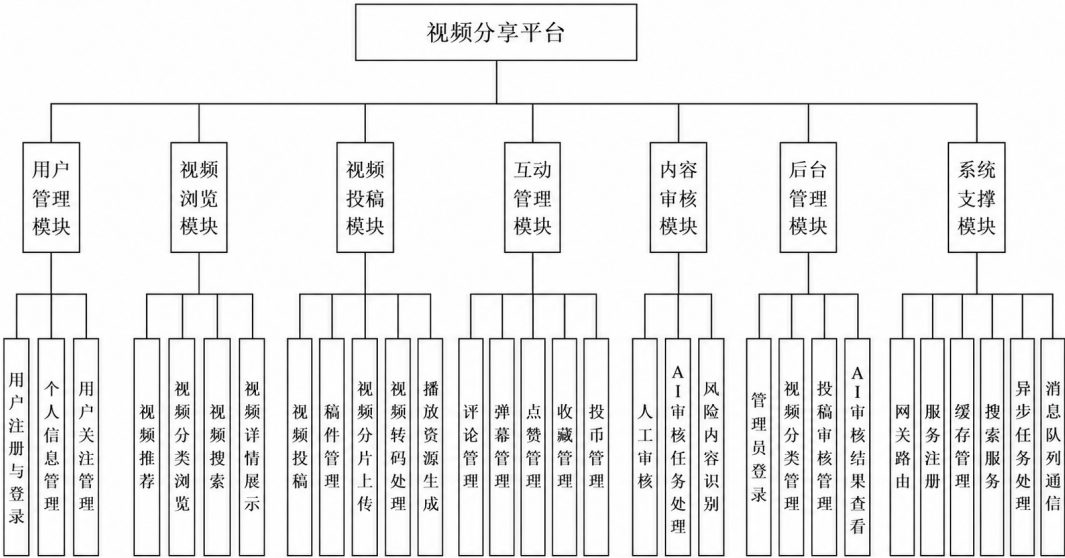


图 3-1 系统功能结构图

### 3.1.2 系统参与角色划分

为了明确系统当中不同用户以及外部服务的权限边界，系统依据业务参与方式以及功能访问范围，把参与角色划分成未登录访问者、注册用户、内容创作者、系统管理员以及 AI 审核服务这五类。系统的用例图如图 3-2 所示。

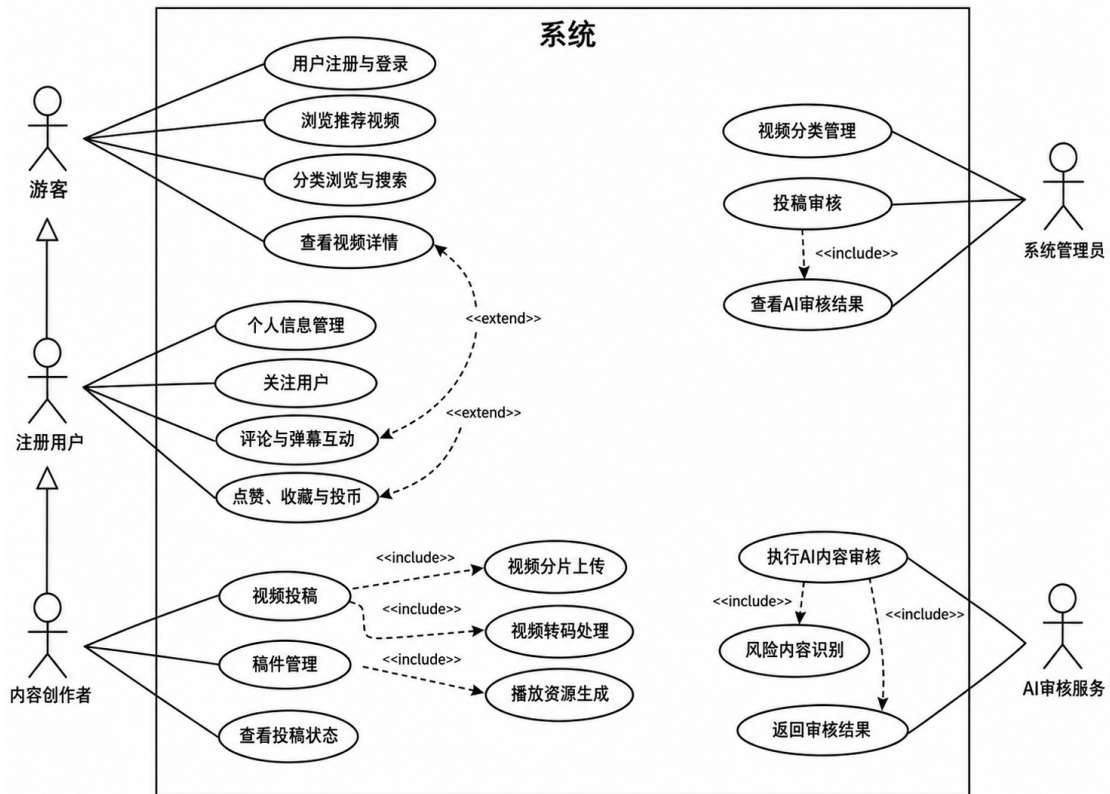


图 3-2 视频分享平台系统用例图

游客是系统当中的基础访问角色，主要使用平台所提供的公开浏览功能。这类用户可进行用户注册与登录、浏览推荐视频、分类浏览与搜索以及查看视频详情等操作，但是不能进行评论、弹幕、点赞、收藏、投币以及投稿这类需要身份认证的行为。

注册用户是在游客基础上完成账号登录之后的用户角色，因此它继承了基础的浏览能力，并且进一步具备了个人信息管理、关注用户、评论与弹幕互动、点赞、收藏以及投币这些功能。注册用户是平台互动行为的主要参与者，其操作有助于增强视频内容传播以及用户之间的交互。

内容创作者是注册用户的一种扩展角色，主要负责视频内容的提交与管理的工作。这类角色具备注册用户的基本功能之外，也可开展视频投稿、稿件管理以及查看投稿状态等操作。在视频投稿过程当中，系统会包含视频分片上传、视频转码处理以及播放资源生成这些子功能，以此保证视频资源可正常存储、处理以及播放。

系统管理员主要负责后台管理与内容监管工作。管理员登录后台后进行视频分类管理、投稿审核以及 AI 审核结果查看等操作。其中，投稿审核过程会包含对 AI 审核结果的查看，管理员结合自动审核的结果来对投稿内容进行人工复核，从而提高内容审核的准确性以及规范性。

AI 审核服务是系统当中的一种外部自动化审核参与者，主要用于辅助完成视频内容的审核工作。这个服务接收到后端系统提交的审核任务之后，会执行 AI 内容审核，并且在审核过程当中完成风险内容识别，最终把审核结果返回给后端系统，从而为管理员的人工审核提供参考依据。

### 3.1.3 系统总体业务流程

在线视频分享平台的核心流程概括为“内容生产、内容处理、内容审核、内容发布、内容消费”这几个环节。视频在公开展示之前需要完成上传、转码以及审核，以此来避免未经处理或者未经审核的内容直接进入前台。

视频平台业务由上传、转码、审核、发布以及消费这多个阶段所组成。其中，上传阶段主要由内容创作者完成，用于提交视频文件、标题、简介、分类、封面这类投稿信息；转码阶段由平台系统负责，会对原始视频文件进行转换，使其变成统一格式以及可供在线播放的资源；审核阶段由 AI 审核服务和管理员共同完成，用于识别涉黄、涉暴、违禁等内容方面的风险，并且为人工复核提供辅助依据；发布阶段是在审核通过之后由系统生成正式的视频记录，然后把视频内容上架到前台页面；消费阶段则面向普通用户，用户可浏览、搜索、播放视频，并且开展评论、弹幕、点赞、收藏以及投币这类互动操作。视频转码和 AI 审核属于比较耗时的任务，因此需要借助状态流转、异步处理以及结果回写保证流程可追踪。数据密集型应用设计当中也强调过，复杂系统需要依靠状态记录以及异常恢复来提升可靠性<sup>[29]</sup>。

系统的总体业务流程如图 3-3 所示。

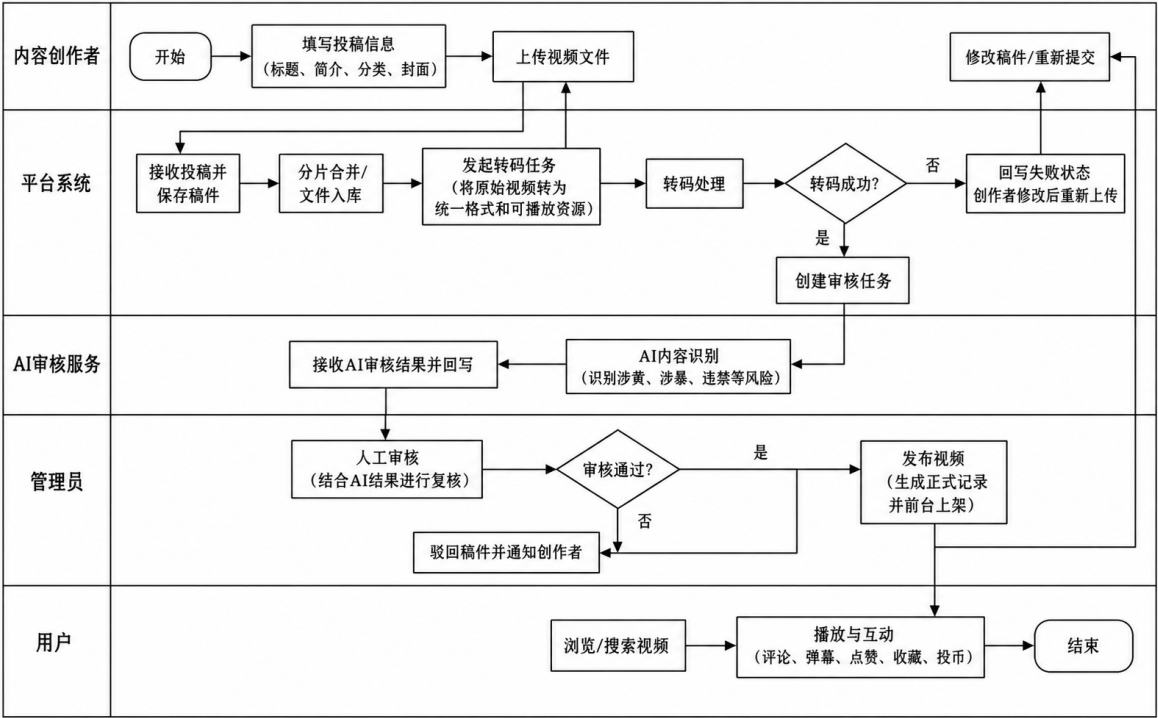


图 3-3 系统总体业务流程图

3.2 用户端功能需求分析

用户端功能公开视频浏览、视频检索、在线播放、社区互动以及账号认证，主要功能需求如表 3-1 所示。

表 3-1 用户端主要功能需求

| 功能类别 | 功能点               | 需求说明              |
|------|-------------------|-------------------|
| 基础浏览 | 首页视频列表、分类筛选       | 用户可查看公开视频内容       |
| 搜索检索 | 关键词搜索、排序、热词       | 用户可根据标题、简介、标签查找视频 |
| 视频播放 | 视频详情、分 P 切换、在线播放  | 用户可稳定播放已发布视频      |
| 用户互动 | 评论、弹幕、点赞、收藏、投币、关注 | 登录用户可参与社区互动       |
| 账号功能 | 注册、登录、退出、自动登录     | 支持用户身份识别和状态维护     |

### 3.2.1 游客访问与用户认证需求

这个平台允许游客浏览公开视频、查看视频详情以及使用搜索功能。但是对于涉及用户身份绑定的操作，例如评论、弹幕、点赞、收藏、投币以及关注这类行为，需用户登录之后才能执行。

注册用户需支持注册、登录、退出以及登录状态校验等功能。登录之后，系统需识别当前用户的身份，并且依据这个身份允许其执行互动操作。普通用户的登录态与管理员的登录态需保持区分，避免后台权限被普通用户误用或者越权访问。权限控制通常需要结合认证与授权机制进行设计，Spring Security 的文档也对认证以及访问控制能力做了一些说明<sup>[31]</sup>。

### 3.2.2 视频浏览、搜索与播放需求

视频浏览是用户端的一项基础功能。系统需支持首页视频列表、分类筛选、视频详情展示以及分 P 播放等功能。首页和分类页面展示每个视频的封面、标题、作者、发布时间以及基础统计数据如收藏数、播放数等，便于用户快速了解视频内容。

搜索功能需支持用户根据关键词查找视频。检索的范围包括视频标题、简介以及标签这些字段，并且要支持按相关性、发布时间或者热度排序。对于内容类的平台来说，搜索功能可弥补单纯首页展示的不足，帮助用户主动发现目标内容。Elasticsearch 这类全文检索技术为复杂的关键词检索提供支持<sup>[12]</sup>。

播放功能需稳定加载已发布视频的播放资源。由于上传的视频通常需要经过转码和切片处理，播放页需支持加载 HLS 资源，并且在播放失败的时候给出明确的提示。HLS 协议会借助播放列表和媒体分片来组织视频资源，比较适合网页端的在线播放场景<sup>[8]</sup>。

### 3.2.3 社区互动需求

社区互动是视频分享平台区别于普通视频播放工具的一个重要特点。系统支持评论、弹幕、点赞、收藏、投币以及关注功能。评论用于表达用户的观点，弹幕与视频的播放时间点相关联，点赞、收藏、投币以及关注则用于记录用户对内容或者作者的偏好。

对于互动行为来说，系统需避免同一个用户重复执行不合理的操作，例如重复点赞或者重复收藏。系统还需在用户再次进入视频详情页的时候展示当前的互动状

态，使用户可清楚看到自己是否已经点赞、收藏或者关注。

### 3.3 创作中心功能需求分析

创作中心主要面向内容创作者，功能需求集中在视频投稿、分片上传、文件处理、稿件状态查看以及作品管理等方面，它的主要功能需求如表 3-2 所示。

表 3-2 创作中心功能需求

| 功能类别 | 功能点               | 需求说明         |
|------|-------------------|--------------|
| 投稿信息 | 标题、分类、标签、简介、封面    | 保存视频基础内容信息   |
| 视频上传 | 分片上传、多文件上传        | 支持大文件稳定上传    |
| 文件处理 | 分片合并、视频转码、播放资源生成  | 使视频满足在线播放要求  |
| 状态查看 | 转码中、待审核、审核通过、审核失败 | 创作者可查看稿件处理进度 |
| 作品管理 | 稿件列表、编辑、删除、统计查看   | 创作者可管理个人投稿内容 |

#### 3.3.1 视频投稿需求

创作中心面向内容创作者，主要承担视频投稿以及作品管理方面的功能。创作者填写视频的标题、选择分类、上传封面、填写标签和简介，并且设置投稿相关的选项。投稿功能的目标要把视频资源、基础信息以及后续的审核流程关联起来。

视频投稿支持多分 P 或者多文件的场景。对于系列视频或者比较长的视频，创作者可能需要上传多个视频片段。系统保存每个分 P 的标题、文件信息以及排序关系，使用户在播放页可切换分 P，同时管理员在审核的时候分别查看不同文件的内容。

#### 3.3.2 视频上传与文件处理需求

视频文件通常体积比较大，若直接一次性上传，容易受到网络波动的影响。因此，系统需支持大文件分片上传。前端把文件拆分成多个分片之后逐个进行上传，服务端接收分片并保存临时文件，等所有分片都上传完成之后再合并。

上传完成之后，系统还需要对视频进行转码处理。不同设备上传的视频格式和编码方式可能不一样，若不统一处理会导致前台播放兼容性较差。FFmpeg 用于完成视频转码、格式转换以及切片这类处理工作<sup>[7]</sup>。



3.3.3 稿件状态与作品管理需求

创作者提交视频之后，需了解稿件当前处于什么阶段，例如转码中、转码失败、待审核、审核通过或者审核不通过。状态的设计尽量清晰，避免使用用户难以理解的内部状态名称。

作品管理功能支持查看稿件列表、查看视频状态、编辑未发布稿件、删除不需要的稿件，以及查看作品基础统计信息。作品管理页面突出投稿时间、审核结果和处理状态，而不是只展示视频封面和播放量。

3.4 管理端与 AI 审核需求分析

管理端与 AI 审核模块主要用于支撑后台认证、分类维护、视频人工审核、AI 审核任务处理和审核结果展示等功能，其主要需求如表 3-3 所示。

表 3-3 管理端与 AI 审核功能需求

| 功能类别    | 功能点                 | 需求说明          |
|---------|---------------------|---------------|
| 管理员认证   | 后台登录、退出、权限校验        | 保证后台功能只对管理员开放 |
| 分类管理    | 分类新增、编辑、删除、排序       | 维护平台内容分类体系    |
| 视频审核    | 审核列表、审核详情、通过、驳回     | 控制视频是否正式发布    |
| AI 审核任务 | 自动创建任务、投递请求、接收结果    | 辅助完成视频内容风险识别  |
| AI 结果展示 | 风险等级、风险标签、命中片段、原因说明 | 帮助管理员快速定位风险内容 |

3.4.1 管理员认证与分类管理需求

管理端主要面向平台的管理员。管理员需通过后台登录之后才能进入管理系统，普通用户不能直接访问后台接口。后台接口需进行身份校验，未登录或者权限不足的时候需拒绝访问。

分类管理是后台管理中的基础功能。视频分类会影响用户投稿时的分类选择，也会影响前台视频展示、筛选和搜索。因此，系统需要支持管理员维护视频分类数据，包括新增分类、修改分类、删除分类、排序和启用停用等操作。



### 3.4.2 人工审核与 AI 审核需求

视频的人工审核是管理端的一个核心需求。创作者提交视频之后，视频不能直接进入正式发布状态，而是需要经过转码以及审核的流程。管理员需查看待审核的视频列表，进入审核详情页，观看视频内容，查看投稿信息以及 AI 审核结果，并且作出通过或者驳回的决定。

AI 审核服务主要负责自动化的风险识别。系统在视频转码完成之后需创建 AI 审核任务，并且把待审核的视频信息发送给 AI 服务。AI 服务完成分析之后，返回审核建议、风险等级、风险标签、命中片段以及原因说明等信息。AI 审核的结果不直接决定视频的最终发布，而是为管理员的人工审核提供辅助依据。人机交互方面的研究当中也指出，AI 系统需为人工决策提供可理解、可纠错的支持，而不是完全替代人工判断<sup>[32]</sup>。

### 3.4.3 视频状态流转需求

为了保证审核流程清晰，系统当中的视频状态需支持流转。投稿视频从用户提交后经历转码中、转码失败、待审核、审核通过或者审核不通过这些状态。视频状态的流转如图 3-4 所示。

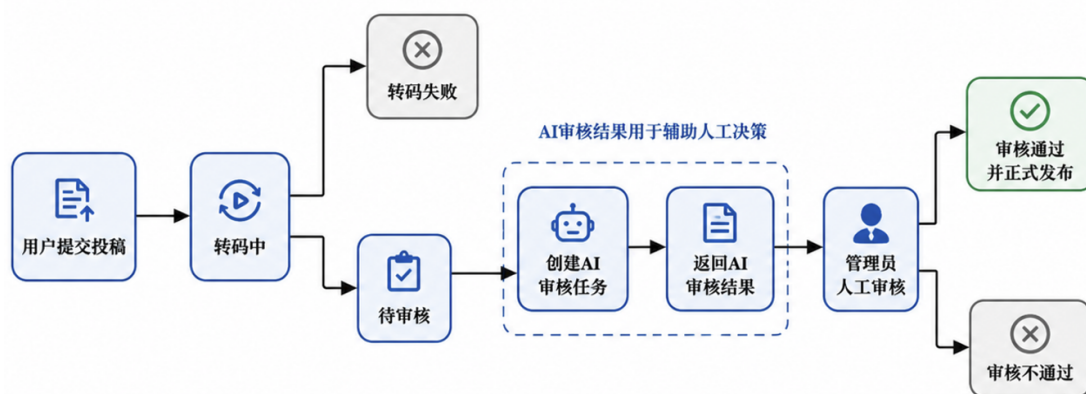


图 3-4 视频投稿与审核状态流转图

由图 3-4 可知，视频是否可发布并不只取决于用户是否提交了投稿，还取决于视频是否转码成功以及管理员是否审核通过。

### 3.5 非功能需求分析

系统的主要非功能需求如表 3-4 所示。

表 3-4 系统主要非功能需求

| 非功能需求 | 需求说明                | 设计关注点           |
|-------|---------------------|-----------------|
| 性能需求  | 常规页面和接口响应保持流畅       | 缓存、搜索、异步任务处理    |
| 可扩展性  | 系统便于后续增加功能和扩展服务     | 模块拆分、服务边界、接口规范  |
| 安全性   | 区分普通用户和管理员权限，保护内部接口 | 身份认证、权限校验、上传限制  |
| 可靠性   | 异常情况下核心流程应尽量不中断     | 状态记录、失败原因、人工处理  |
| 数据一致性 | 投稿、转码、审核和发布状态需保持一致  | 事务控制、状态流转、结果回写  |
| 可维护性  | 系统结构清晰，便于后续修改和扩展    | 分层设计、公共模块、日志和文档 |

非功能需求主要关注系统的质量属性。在性能、可扩展性、安全性、可靠性、一致性以及可维护性方面满足基本的要求。

在性能方面，视频列表、评论加载和搜索查询等常规接口可保持较好的响应速度；视频上传、视频转码及 AI 审核耗时任务采用异步处理的方式，避免用户长时间等待。ISO/IEC 25010 软件质量模型将性能效率作为软件质量的一个重要方面<sup>[33]</sup>。

系统可扩展性方面，当业务需求增大时可扩展不同的模块，例如资源处理压力比较大的时候优先优化资源服务，互动访问比较频繁时优先优化互动模块。安全性方面，系统需区分普通用户和管理员的权限，对后台接口、上传文件以及审核接口进行保护。

可靠性和一致性方面，系统需保证视频投稿、文件处理、审核状态以及正式发布数据之间保持逻辑上的一致性。对于涉及多个服务协作的关键写操作，系统需具备事务协调或者状态补偿的能力；对于视频转码、AI 审核以及播放统计这类耗时或者高频的任务，则需借助异步处理以及结果回写保证最终状态一致。若转码失败、AI 审核失败或者服务短暂不可用，系统需记录状态以及失败原因，避免业务状态出现混乱。可维护性方面，系统需保持清晰的模块边界、接口规范以及日志记录，便于后续修改以及问题排查。

## 第4章 系统架构及数据库设计

### 4.1 系统总体架构设计

#### 4.1.1 系统总体架构

系统总体架构如图 4-1 所示。

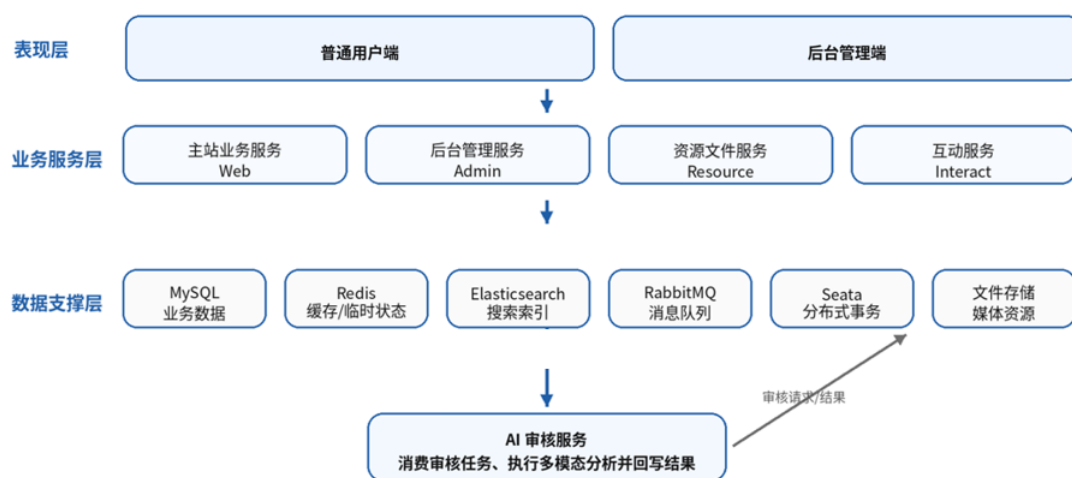


图 4-1 系统总体架构图

图 4-1 中的架构是围绕用户访问、业务处理、资源处理、AI 审核和数据支撑等部分进行分层设计。各层之间通过网关转发、服务调用、消息队列和数据存储组件进行协作，从而支撑视频投稿、转码、审核、发布、播放和互动等核心业务流程。

**（1）表现层。**表现层可分成普通用户端和后台管理端。普通用户端提供视频浏览、关键词搜索、视频播放、投稿上传、评论弹幕、点赞收藏等功能入口；后台管理端提供分类管理、视频审核、AI 审核结果查看、内容管理等功能入口。表现层只需将请求通过网关转发给后端服务处理复杂业务逻辑。

**（2）业务服务层。**业务服务层是系统核心业务处理部分，业务服务分成了主站业务服务、后台管理服务、资源文件服务和互动服务。主站业务服务负责用户信息、首页数据、视频信息、投稿信息、搜索编排和 AI 审核任务创建等业务；后台管理服务负责管理员登录、分类维护、视频审核和审核结果查看；资源文件服务负责封面图片上

传、视频分片上传、分片合并、视频转码和 HLS 播放资源输出文件处理等；互动服务负责评论、弹幕、点赞、收藏和投币等高频互动行为。服务拆分可避免所有业务都集中于单个服务，同时降低模块之间的耦合度。

(3) **AI 能力层**。AI 能力层负责处理 AI 视频的审核。在视频完成转码后，主站业务服务会创建审核任务，将任务交给消息队列向 AI 审核服务发送审核请求。AI 审核服务消费任务后会对视频内容进行分析，并将审核结果返回给后端业务服务。AI 服务不会被某个服务直接内部调用，而是通过异步方式与后端服务通信。并且服务返回的结果是为管理员人工审核提供辅助判断依据，由后台审核流程控制。

(4) **数据支撑层**。数据支撑层为系统运行提供基础数据能力，主要包括 MySQL、Redis、Elasticsearch、RabbitMQ、Seata 和本地文件目录等。MySQL 用于存储核心业务数据；Redis 用于保存临时状态、缓存数据和轻量队列数据；Elasticsearch 则用于建立视频搜索索引，提高关键词检索能力；RabbitMQ 支撑 AI 审核任务的异步投递和结果回传；Seata 支撑跨服务写操作的数据一致性；本地文件目录用于保存封面、上传分片、转码视频、HLS 切片和 AI 审核截图等媒体资源。

## 4.2 微服务模块划分

### 4.2.1 系统服务模块说明

表 4-1 系统服务模块及职责

| 模块名称                   | 网关前缀      | 主要职责                        |
|------------------------|-----------|-----------------------------|
| streama-cloud-gateway  | —         | API 统一入口、路由转发、内部接口隔离、管理端过滤  |
| streama-cloud-web      | /web      | 用户、首页、视频信息、投稿、搜索和 AI 审核任务编排 |
| streama-cloud-admin    | /admin    | 管理端登录、分类管理、视频审核、审核结果查询      |
| streama-cloud-resource | /file     | 图片上传、视频分片上传、视频转码和 HLS 资源输出  |
| streama-cloud-interact | /interact | 评论、弹幕、点赞、收藏等互动行为处理          |
| streama-cloud-common   | —         | Redis、搜索、异常处理、公共组件和工具类      |
| streama-cloud-base     | —         | 常量、枚举、基础实体和公共 DTO           |
| ai-service             | —         | 消费审核任务，执行多模态审核返回审核结果        |

系统服务模块及其职责如表 4-1 所示。

主站服务对应 web 模块，承担核心业务处理职责，但不是所有功能的承载点。文件处理交由资源服务模块 resource 管理；用户与用户之间的互动如评论、弹幕等拆分给互动服务模块 interact。后台服务模块 admin 独立处理管理端接口，使管理员接口和普通用户接口更容易区分。common 模块和 base 模块提供跨服务的公共组件和基础数据结构，避免重复开发。AI 服务模块独立部署，专注于 AI 审核任务处理。

#### 4.2.2 服务协作关系

各服务之间围绕着具体业务场景协作。当用户提交投稿信息时，主站服务负责保存投稿主数据和分 P 文件数据，主数据为视频的标题，简介等信息；P 文件数据代表这个视频可能包含多个视频文件，以待转码文件的形式加入队列；资源服务消费队列后完成视频转码，并回写转码结果；当全部分 P 转码完成后，主站服务会创建 AI 审核任务并发送消息；AI 服务审核完成后回传结果；管理员最终在后台完成复核与发布操作。

表 4-2 系统主要服务调用关系

| 调用方  | 被调用方          | 调用方式 | 主要用途              |
|------|---------------|------|-------------------|
| 网关服务 | 主站/后台/资源/互动服务 | 路由转发 | 将外部请求转发到对应服务      |
| 主站服务 | 后台服务          | 同步调用 | 获取分类树和分类列表        |
| 主站服务 | 互动服务          | 同步调用 | 查询用户互动状态,联动处理互动数据 |
| 资源服务 | 主站服务          | 内部接口 | 回写视频转码结果和文件信息     |
| 主站服务 | AI 审核服务       | 消息队列 | 发送审核请求并接收审核结果     |
| 后台服务 | 主站服务          | 内部接口 | 查询待审核视频并提交人工审核结果  |

由表 4-1 对服务与服务之间调用关系和用途进行简单的说明，这种调用关系将同步请求和异步任务进行了区分。查询分类、查询互动状态等轻量业务采用同步调用；视频转码、AI 审核等耗时任务则使用队列异步处理，避免用户请求长时间阻塞。



（4）`video_info_post` 与 `video_info_file_post`：投稿态数据表组，分别存储待审视频的主信息与分 P 文件及转码状态。其中 `status`、`transfer_result` 等控制字段专用于投稿流程管理，内容不进入公开访问域。

（5）`video_info` 与 `video_info_file`：正式发布数据表组，存储审核通过后的视频主信息与播放资源文件。相较投稿表，增加了播放量、互动计数等前台展示所需字段，直接服务于搜索、推荐和在线播放。

（6）`video_comment`：视频评论表。利用 `p_comment_id` 实现回复层级，包含点赞点踩计数及置顶标记，支撑评论区完整交互。

（7）`video_danmu`：视频弹幕表。记录文本内容、视频内出现时间点及样式信息，服务于弹幕的实时发送与加载。

（8）`user_action`：用户行为记录表。通过 `action_type` 区分点赞、收藏、投币等操作，为前台展示和后台行为分析提供明细数据。

为保证未审核内容与公开链路的严格隔离，系统在数据层面采用了“先入投稿库，后迁正式库”的分离设计。

创作者提交后，视频主信息和分 P 文件状态被分别写入 `video_info_post` 与 `video_info_file_post`，此后经历转码与审核。这期间的投稿态数据仅供内部流程控制，对用户不可见。一旦审核通过，系统自动将核心数据迁移至 `video_info` 与 `video_info_file` 等正式发布表，并同步构建前台索引，使视频正式上线。

这一“投稿态—正式态”分离设计，天然将面向流程的写入/更新操作，与面向用户的公开读取操作解耦。它既保障了内容合规审核的数据安全，避免待审或违规内容的外泄，也让前台公开数据保持相对稳定，利于提升检索效率与播放稳定性。同时，各阶段的独立状态控制，增强了业务流程的可管理性和系统的整体可维护性。

#### 4.3.2 AI 审核数据设计

AI 审核数据采用主从结构。`ai_audit_task` 保存视频级审核任务，例如请求编号、视频编号、审核版本、任务状态、AI 建议、风险等级和审核摘要等信息；`ai_audit_task_item` 保存分 P 级审核结果，例如文件编号、风险分数、风险标签、命中片段和原因说明等信息。

采用主从结构的原因是，一个视频可能包含多个分 P 文件，AI 审核既需要给出整条视频的综合结论，也需要保留每个分 P 的风险细节。如果只保存一个总结果，管

理员难以定位风险出现在哪个视频片段；如果只保存明细而没有总结果，审核列表展示又不够直观。

表 4-3 AI 审核数据设计

| 数据表                | 数据粒度      | 主要字段含义                              |
|--------------------|-----------|-------------------------------------|
| ai_audit_task      | 视频级任务     | 请求编号、视频编号、审核版本、任务状态、AI 建议、风险等级、审核摘要 |
| ai_audit_task_item | 分 P 文件级结果 | 文件编号、风险分数、风险标签、命中片段、关键帧路径、原因说明      |

#### 4.4 消息通信与 AI 审核任务设计

视频上传、视频转码、AI 审核等操作属于耗时较长的业务流程。如果这些任务在用户请求过程中同步执行，会导致接口响应时间变长，影响用户投稿体验。因此在视频投稿与审核流程中引入异步任务处理机制，将耗时操作从主业务流程中拆分出来，通过消息通信和任务状态管理提高系统的稳定性。

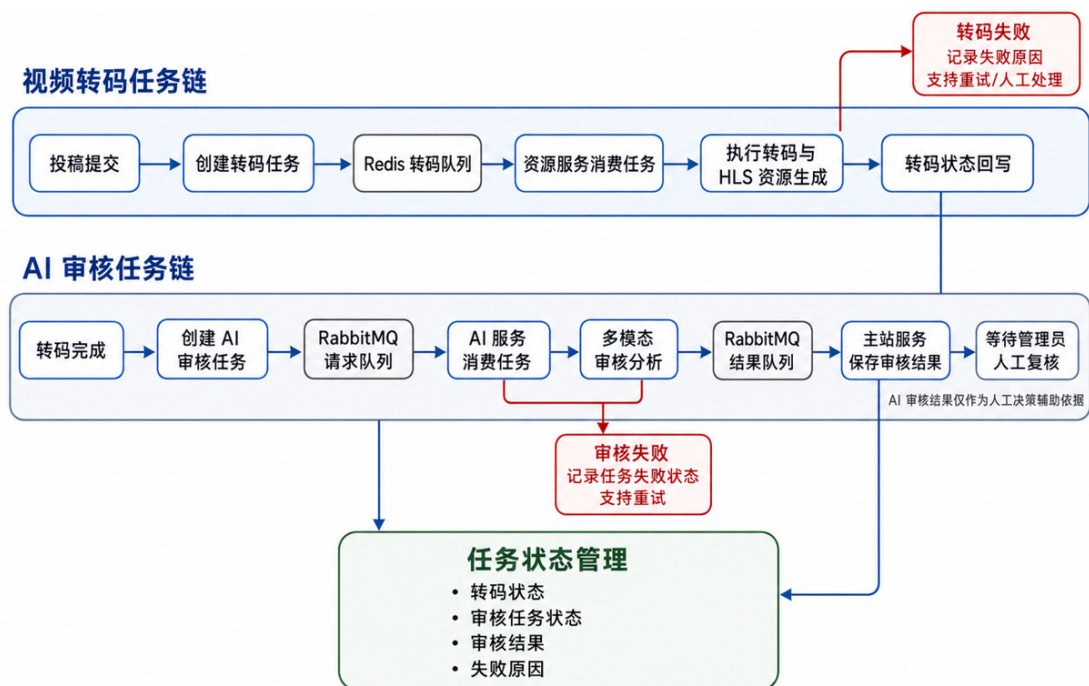


图 4-3 异步任务总体流程图



4.4.1 消息通信设计

如图 4-3 所示，系统的异步任务主要围绕视频投稿后的文件处理流程展开。用户提交视频后，系统先保存投稿态数据以及记录视频文件的基础信息。由于视频文件转码、状态更新和结果回写等操作耗时较长，将相关任务交由后台异步执行。这样可缩短用户投稿接口的响应时间，同时避免长时间同步处理造成接口阻塞。

消息通信设计的核心目标是实现业务流程解耦。投稿接口只负责完成必要的数据保存和任务创建，后续的文件处理、转码执行和状态更新则由独立的后台任务完成。不同服务之间通过消息或任务记录传递必要的信息，例如视频编号、文件编号、文件路径、任务状态等。处理服务获取任务后，根据任务内容执行对应操作，并在处理完成后更新数据库中的任务状态和业务状态。

任务状态是异步流程控制的重要依据。由于异步任务并不是创建后立即完成，系统需要记录任务在不同阶段的执行情况，例如待处理、处理中、处理成功和处理失败等。通过状态字段，系统可判断当前任务是否已经被执行、是否执行成功、是否需要重新处理，以及是否进入下一业务环节。

表 4-4 异步任务状态设计

| 状态   | 含义           | 处理说明                |
|------|--------------|---------------------|
| 待处理  | 任务已创建但尚未开始执行 | 等待后台任务消费或定时任务扫描处理   |
| 处理中  | 任务已被获取并正在执行  | 记录任务执行过程，避免重复处理     |
| 处理成功 | 任务正常完成并返回结果  | 更新业务数据，进入下一流程节点     |
| 处理失败 | 任务执行过程出现异常   | 记录失败原因，必要时进行重试或人工处理 |

在异常处理方面，系统需要考虑文件处理失败、转码失败、状态更新失败以及任务长时间未完成等情况。对于具备自动恢复条件的异常任务，系统通过重试机制重新执行；对于多次处理失败或无法自动判断的问题，则需要记录异常原因，并交由管理员进行人工处理。通过任务状态和异常信息的配合，系统可追踪异步任务的执行过程，避免异常任务影响正常业务流程。

#### 4.4.2 AI 审核消息通道设计

AI 审核任务采用 RabbitMQ 进行消息通信。主站服务作为审核请求生产者，将视频编号、分 P 文件信息、资源路径和任务编号等信息发送到审核请求队列；AI 服务作为消费者，从队列中获取任务并执行审核；审核完成后，AI 服务将结果发送到审核结果队列，主站服务再消费结果并写入数据库。

表 4-5 AI 审核消息通道设计

| 消息组件     | 名称                          | 作用              |
|----------|-----------------------------|-----------------|
| Exchange | streama.audit.exchange      | AI 审核请求和结果共用交换机 |
| 请求队列     | streama.audit.request.queue | AI 服务消费审核请求     |
| 请求路由键    | audit.video.request         | 主站服务发送审核请求时使用   |
| 结果队列     | streama.audit.result.queue  | 主站服务消费审核结果      |
| 结果路由键    | audit.video.result          | AI 服务发送审核结果时使用  |
| 请求死信队列   | streama.audit.request.dlq   | 保存请求处理失败的消息     |
| 结果死信队列   | streama.audit.result.dlq    | 保存结果处理失败的消息     |

该消息通道设计包含请求投递、任务消费、结果回传和失败处理。即使 AI 服务暂时不可用，请求消息也可保留在队列中，等待服务恢复后继续处理。

#### 4.5 视频业务流程设计

视频业务设计流程划分为投稿与资源处理、审核与发布、消费与互动三个阶段。视频业务服务协作流程如图 4-4 所示。

##### 4.5.1 视频投稿与资源处理流程设计

视频投稿流程由内容创作者发起，涉及主站服务和资源服务之间的协作。创作者在前端填写视频标题、分类、标签、简介、封面和分 P 文件信息后，系统首先对投稿基础信息进行保存。由于视频文件通常体积较大，资源上传不适合采用普通表单一次性提交，因此系统采用分片上传方式处理视频文件。前端在上传前向资源服务申请上传标识，资源服务根据上传标识保存分片临时文件，并在全部分片上传完成后进行文件合并。

投稿信息与视频资源不是一次性同步完成的。主站服务负责保存视频投稿主数

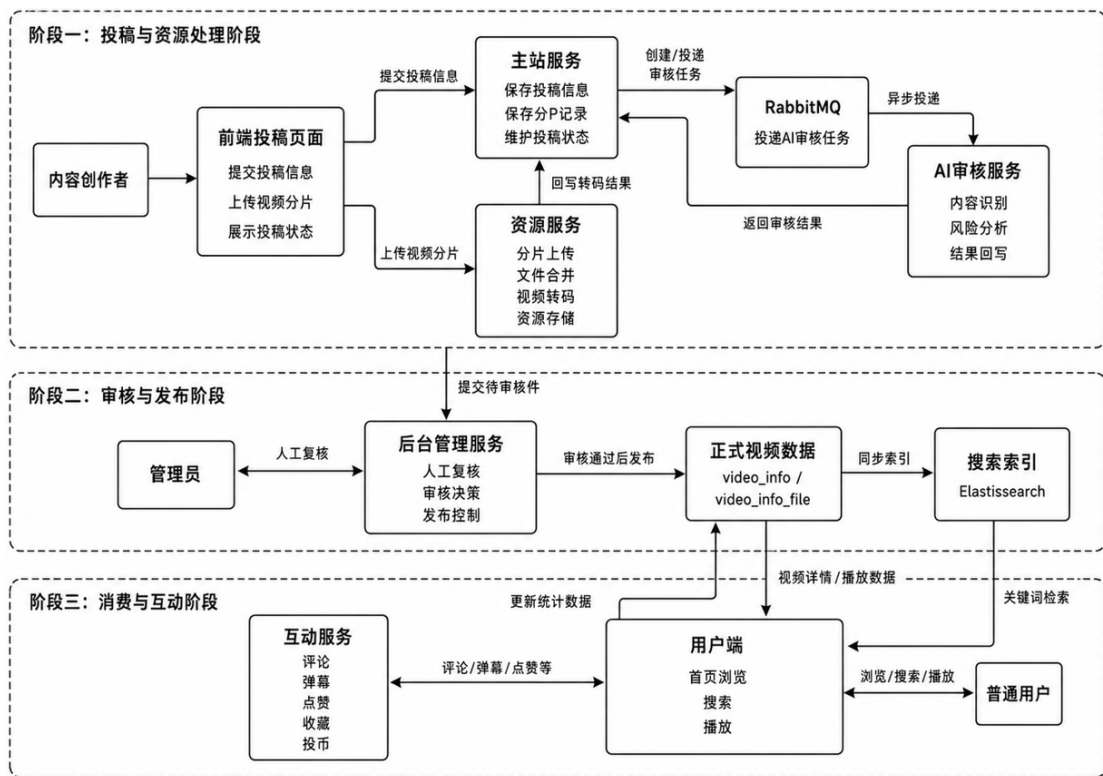


图 4-4 视频业务流程设计图

据和分 P 文件记录，资源服务负责处理文件上传、合并、转码和播放资源生成。视频文件转码完成后，资源服务将处理结果回写给主站服务，主站服务再根据各分 P 文件的处理结果判断投稿是否进入后续审核阶段。该设计可避免主站服务直接承担大文件处理任务，也能使视频转码失败时保留明确的处理状态。

该流程中的投稿数据仍处于待处理或待审核状态，不会直接进入前台公开视频列表。

#### 4.5.2 视频审核与发布流程设计

视频完成资源处理后，需要进入审核与发布阶段。系统采用 AI 审核辅助和管理员人工复核结合的方式。主站服务在确认视频文件转码完成后，创建 AI 审核任务，并通过消息队列将视频编号、分 P 文件信息和资源路径发送给 AI 审核服务。AI 审核服务消费任务后，对视频内容进行分析，并将风险等级、命中标签、命中片段和审核建议等结果返回给主站服务。

AI 审核结果只作为管理员审核的辅助依据，不直接决定视频最终是否发布。管理员在后台审核页面查看投稿信息、视频内容和 AI 审核结果后，选择审核通过或审

核不通过。审核通过时，系统将投稿态视频数据转换为正式发布数据，并同步写入正式视频表、正式视频文件表和搜索索引；审核不通过时，系统保留投稿状态和审核结果，避免不符合要求的视频进入用户端展示范围。

这种设计将“自动分析”和“最终决策”分开处理。一方面，AI 审核可提前发现疑似风险内容，降低管理员审核压力；另一方面，人工复核可避免完全依赖自动审核带来的误判问题。视频发布结果最终由后台审核流程控制，从而保证内容发布流程的可控性。

### 4.5.3 视频消费与互动流程设计

视频审核通过并正式发布后，普通用户可在前台进行浏览、搜索和播放。系统前台展示的数据主要来自正式视频数据，而不是投稿态数据。首页推荐、分类筛选和视频详情页只展示已经通过审核的视频内容，搜索功能则通过 Elasticsearch 索引提高关键词检索效率。这样可避免待审核、审核失败或转码失败的视频被普通用户访问。

视频播放过程中，用户可根据视频分 P 信息选择不同片段进行观看。播放资源由资源服务提供，视频基础信息、作者信息和统计信息由主站服务提供。对于评论、弹幕、点赞、收藏、投币等高频互动行为，系统由互动服务单独处理，避免所有操作都集中在主站服务中。互动结果通过统计字段或异步更新方式反映到视频展示数据中。

视频消费流程与投稿审核流程相比，更强调读取效率和用户体验。投稿审核阶段关注的是状态控制和内容安全，视频消费阶段关注的是公开视频的快速查询、稳定播放和互动数据维护。通过将视频投稿、审核发布和用户互动划分为不同流程，系统在总体设计上保持较清晰的业务边界。

## 第5章 视频分享平台的设计与实现

用户端功能模块是普通用户访问系统的主要入口，包括视频浏览、视频搜索、视频播放、用户认证和互动操作等功能。该模块并不是单一的数据读取模块，而是由主站业务服务、互动服务、缓存组件、检索组件和数据库共同协作完成。公开视频浏览、分类查询和视频搜索允许未登录用户访问，而点赞、收藏、投币、关注、评论和发送弹幕等操作需要用户完成登录认证后才能执行。

### 5.1 用户端功能模块实现

#### 5.1.1 用户端整体功能实现

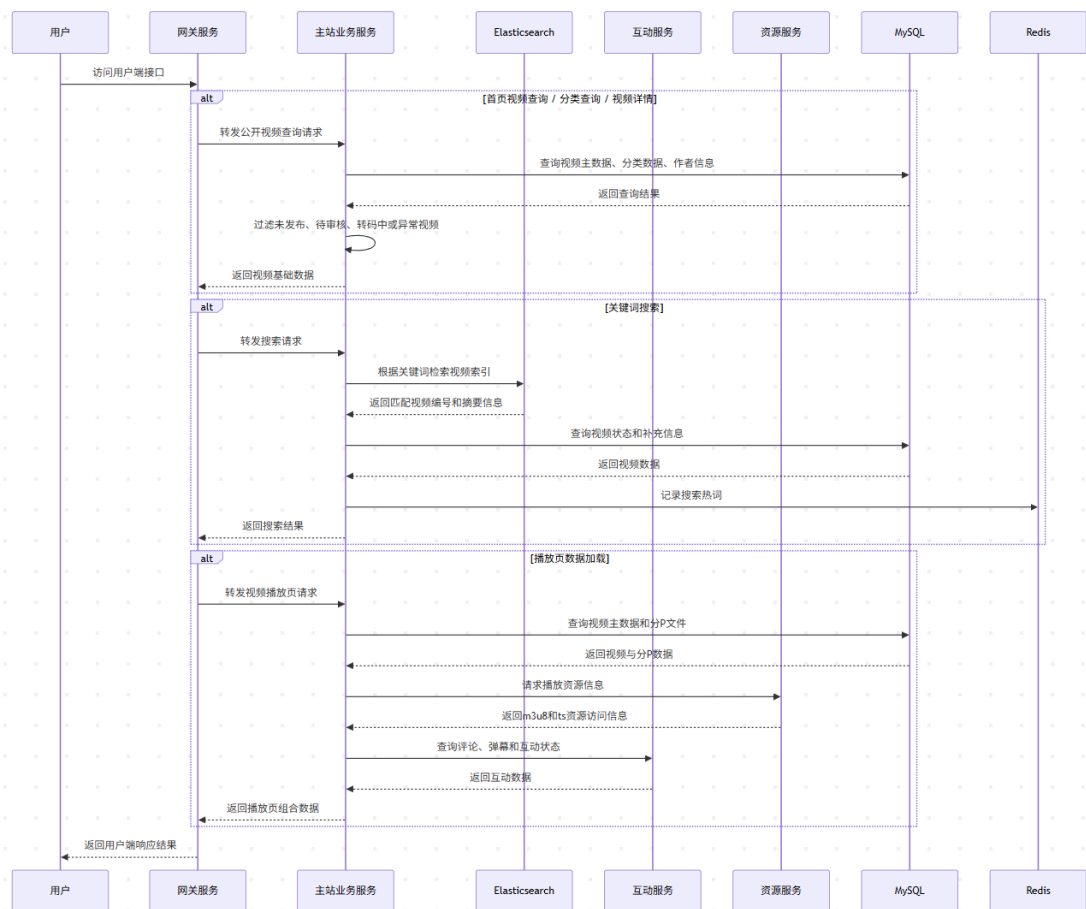


图 5-1 用户端核心接口处理时序图

用户端功能模块主要给用户展示公开视频的数据读取、互动状态判断和登录校验等逻辑，系统如何完成视频列表查询、搜索结果返回、播放页数据组合等处理。

首页的视频列表、分类查询、关键词搜索以及视频详情读取属于公开读取接口，公开读取接口允许未登录用户访问，但会统一过滤未发布、待审核、转码中或资源异常的视频数据。实行点赞、收藏、投币、关注、评论和弹幕发送属于登录写入类接口，系统会先解析请求令牌，并从 Redis 中读取用户登录信息；如果登录态不存在或已经过期，则直接返回未登录结果，不执行后续操作。

用户端核心接口处理过程如图 5-1 所示。系统在接收到请求后则通过网关转发到对应业务服务。视频基础数据主要由主站业务服务查询，搜索请求结合 Elasticsearch 完成关键词匹配，评论、弹幕和用户行为由互动服务处理，播放资源则通过资源服务读取。各服务处理完成后，系统将视频主数据、分 P 文件、播放资源和互动状态封装为统一响应结果返回。

### 5.1.2 首页浏览与视频搜索实现

首页浏览与视频搜索接口都属于公开视频读取接口，但二者在代码中的入口和查询方式并不相同。首页浏览接口主要对应 `/web/video/loadVideoList`，该接口根据父级分类编号、分类编号、页码和分页大小构造 `VideoInfoQuery` 查询对象，并通过 `VideoInfoService` 返回分页视频列表。关键词搜索接口主要对应 `/web/video/search`，该接口接收关键词、排序类型和页码参数，先将关键词写入 Redis 热词统计，再通过 `EsSearchComponent` 完成搜索并返回分页结果。

首页浏览与视频搜索接口说明如表 5-1 所示。

表 5-1 首页浏览与视频搜索接口说明

| 接口路径                                        | 主要参数                                                                                                   | 实现说明                                                                                                 |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <code>/web/video/loadVideoList</code>       | <code>pCategoryId</code> 、<br><code>categoryId</code> 、<br><code>pageNo</code> 、 <code>pageSize</code> | 根据分类和分页条件构造 <code>VideoInfoQuery</code> ，调用 <code>VideoInfoService.findListByPage</code> 查询正式视频分页数据。 |
| <code>/web/video/search</code>              | <code>keyword</code> 、<br><code>orderType</code> 、 <code>pageNo</code>                                 | 对关键词进行非空校验后写入 Redis 热词统计，并调用 <code>EsSearchComponent.search</code> 返回搜索分页结果。                         |
| <code>/web/video/getSearchKeywordTop</code> | 无                                                                                                      | 从 Redis 中读取搜索热词列表，用于搜索页或首页搜索提示展示。                                                                    |

首页浏览与视频搜索处理过程如图 5-2 所示。系统首先判断请求中是否包含关键词：若不包含关键词，则按分类和分页条件查询 MySQL 中的公开视频数据；若包含关键词，则先通过 Elasticsearch 检索视频标题和标签等字段，同时将关键词写入 Redis 搜索热词统计。两条路径取得候选视频后，都会继续过滤视频状态，并补充作者、分类和统计数据，最后封装为视频摘要列表返回给用户端。



图 5-2 首页浏览与视频搜索处理流程图

首页浏览与视频搜索实现过程如算法 1 所示。

#### 算法 1 首页浏览与视频搜索实现算法

输入： 分类条件  $C$ ，关键词  $keyword$ ，排序类型  $orderType$ ，页码  $pageNo$

输出： 公开视频分页结果  $P$

```

1: if  $keyword$  为空 then
2:   构造 VideoInfoQuery 查询对象  $Q$ 
3:   将父级分类、子分类、页码和分页大小写入  $Q$ 
4:   设置  $Q$  只查询公开视频，并按创建时间倒序排序
5:   调用 VideoInfoService.findListByPage 查询 MySQL，得到分页结果  $P$ 
6: else
7:   调用 RedisComponent.addKeywordCount 记录搜索热词
8:   根据  $keyword$ 、 $orderType$  和  $pageNo$  构造 Elasticsearch 检索条件
9:   调用 EsSearchComponent.search 检索视频标题和标签字段，得到分页结果  $P$ 
10: end if
11: for 每一个视频摘要  $v \in P.list$  do
12:   过滤不可公开访问的视频状态
13:   补充作者昵称、分类信息和播放量、弹幕量等统计字段
14: end for
15: 将总记录数、当前页、总页数和视频摘要列表封装到 PaginationResultVO
16: return  $P$ 
  
```

算法 1 对应的前端实现效果如图 5-3 所示。用户在首页顶部搜索框输入关键词“低头”并提交后，系统进入搜索结果展示模式，页面显示当前搜索关键词、排序条件以及匹配到的视频列表。视频卡片中展示了视频封面、标题、作者信息以及播放



图 5-3 首页浏览与视频搜索结果界面

量、弹幕量、收藏量等摘要数据，右上方的播放量、发布时间、弹幕量和收藏量按钮对应搜索接口中的排序参数，用于控制搜索结果的展示顺序。该界面说明后端返回的分页视频数据已经被前端统一渲染为可浏览、可排序、可分页的视频搜索结果。

首页浏览和关键词搜索虽然查询入口不同，但最终都使用 `PaginationResultVO` 封装分页结果，便于前端以统一方式渲染视频列表。接口返回的数据以 `VideoInfo` 为主要内容对象，通常包含视频编号、标题、封面、分类、作者、播放数、弹幕数、点赞数、收藏数和发布时间等展示字段。系统只允许正式发布且资源可访问的视频进入公开视频列表，未发布、待审核、转码中或资源异常的视频不会被返回。搜索热词只用于 Redis 统计用户搜索行为，不直接参与公开视频过滤。

### 5.1.3 视频播放页实现

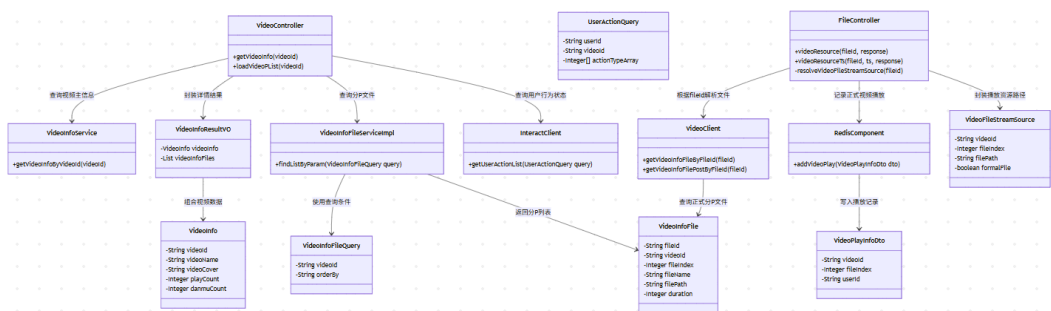
视频播放页实现主要围绕视频详情、分 P 文件、用户行为状态和播放资源读取几个接口展开。与用户端整体接口时序不同，本节不再重复说明网关转发和服务协作过程，而是从播放页所依赖的接口和关键数据对象角度说明其实现方式。

视频播放页进入时，首先根据视频编号调用 `/web/video/getVideoInfo` 接口读取视频主信息。该接口内部通过 `VideoInfoService.getVideoInfoByVideoId` 查询正式视频数据；如果视频记录不存在，则直接抛出业务异常。若当前请求中能够解析出登录用户信息，系统还会构造 `UserActionQuery`，并通过互动服务查询当前用户对该视频的点赞、收藏和投币状态，最后将视频主信息和用户行为状态封装为



视频播放页相关接口说明如表 5-2 所示。

| 接口路径                                  | 主要参数      | 实现说明                                        |
|---------------------------------------|-----------|---------------------------------------------|
| /web/video/getVideoInfo               | videoId   | 查询视频主信息，并在用户已登录时补充点赞、收藏、投币等行为状态。            |
| /web/video/loadVideoPList             | videoId   | 根据视频编号查询分 P 文件列表，并按照 file_index asc 排序返回。   |
| /resource/videoResource/{fileId}      | fileId    | 根据分 P 文件编号读取 HLS 的 m3u8 索引文件；正式视频访问时写入播放记录。 |
| /resource/videoResource/{fileId}/{ts} | fileId、ts | 根据分 P 文件编号和切片名称读取对应的 ts 切片文件。               |



如图 5-4 所示,播放页接口涉及主站业务服务和资源服务两部分。VideoController 负责提供视频详情和分 P 列表数据, VideoInfoResultVO 用于组合视频主信息和用户行为状态, VideoInfoFileQuery 用于按视频编号查询分 P 文件。FileController

负责根据 `fileId` 读取播放资源，系统会先通过 `VideoClient` 查询正式分 P 文件；如果未查询到正式文件，再查询投稿态分 P 文件，用于支持预览类场景。

播放资源并不直接暴露服务器真实目录。播放页只持有分 P 文件编号，资源服务根据 `fileId` 解析文件路径后读取 `index.m3u8` 和对应的 `ts` 切片文件。对于正式视频，资源服务在读取 `m3u8` 时会构造 `VideoPlayInfoDto`，并将视频编号、分 P 序号和可选的用户编号写入 Redis 播放记录队列。这样既能保证播放资源由统一接口输出，也能为后续播放量统计提供数据来源。

## 5.2 创作中心与视频投稿实现

创作中心与视频投稿实现主要围绕投稿态数据处理展开。项目中创作中心相关接口集中在 `CenterVideoPostController` 中，其访问路径以 `/ucenter` 为前缀，并通过登录拦截保证投稿数据与当前用户绑定。资源上传相关接口由资源服务中的 `FileController` 提供，投稿保存接口并不直接接收完整视频文件，而是接收视频基础信息和分 P 文件列表，其中分 P 文件通过 `uploadId` 与前置上传任务关联。

### 5.2.1 创作中心功能结构

创作中心功能结构在代码中主要体现为一组围绕投稿态视频的接口。相关接口说明如表 5-3 所示。

表 5-3 创作中心投稿相关接口说明

| 接口路径                                       | 主要参数                                                                    | 实现说明                              |
|--------------------------------------------|-------------------------------------------------------------------------|-----------------------------------|
| <code>/ucenter/loadVideoPostList</code>    | <code>status</code> 、 <code>pageNo</code> 、 <code>videoNameFuzzy</code> | 根据当前用户编号、稿件状态和标题模糊条件查询投稿态视频分页数据。  |
| <code>/ucenter/getVideoCountInfo</code>    | 无                                                                       | 统计当前用户审核通过、审核失败和处理中稿件数量。          |
| <code>/ucenter/getVideoByVideoId</code>    | <code>videoId</code>                                                    | 校验视频归属后，读取投稿主信息和分 P 文件列表，用于编辑页回显。 |
| <code>/ucenter/postVideo</code>            | 投稿基础信息、 <code>uploadFileList</code>                                     | 保存投稿主信息和分 P 文件信息，并根据文件变化初始化稿件状态。  |
| <code>/ucenter/saveVideoInteraction</code> | <code>videoId</code> 、 <code>interaction</code>                         | 保存视频评论、弹幕等互动开关配置。                 |
| <code>/ucenter/deleteVideo</code>          | <code>videoId</code>                                                    | 根据视频编号和当前用户编号删除稿件或公开视频数据。         |

稿件列表查询接口根据当前登录用户编号构造 `VideoInfoPostQuery`，并按照创建时间倒序返回分页结果；稿件数量统计接口分别统计审核通过、审核失败和处理中稿件数量；稿件详情接口根据视频编号读取 `VideoInfoPost` 和对应的 `VideoInfoFilePost` 列表，并返回给编辑页面。

创作中心的核心对象关系如图 5-5 所示。`CenterVideoPostController` 负责接收创作中心请求，并将用户编号写入查询对象或投稿对象中；`VideoInfoPostService` 负责处理投稿主记录保存、分页查询和状态更新；`VideoInfoFilePostService` 负责读取分 P 文件列表；`VideoPostEditInfo` 用于组合投稿主信息和分 P 文件信息，返回给编辑页面。

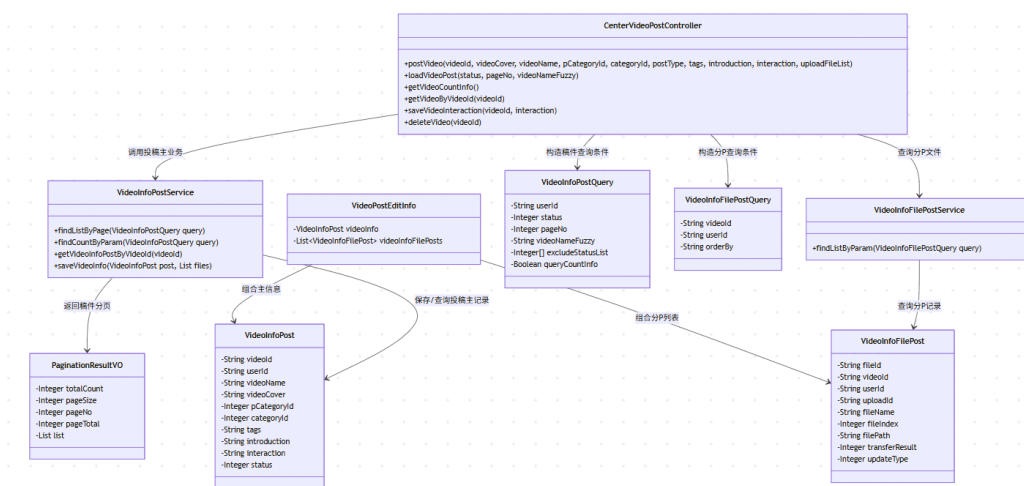


图 5-5 创作中心投稿关键类图

在数据层面，创作中心主要操作投稿态表，而不是直接操作正式公开视频表。`VideoInfoPost` 保存投稿视频的标题、封面、分类、标签、简介、互动配置和投稿状态等信息；`VideoInfoFilePost` 保存每个分 P 文件的上传标识、文件编号、文件名称、分 P 顺序、转码结果和文件路径等信息。一个投稿视频可以对应多个分 P 文件，因此编辑页面需要同时读取这两类数据。

### 5.2.2 视频分片上传实现

视频分片上传由资源服务处理，上传前，系统根据文件名和分片数量创建上传任务，并在 Redis 中保存 `UploadFileDto`；上传过程中，每个分片请求都携带 `uploadId` 和 `chunkIndex`，资源服务根据当前用户编号和 `uploadId` 读取 Redis 中的上传任务，

再将分片写入临时目录。

视频分片上传相关接口说明如表 5-4 所示。

表 5-4 视频分片上传接口说明

| 接口路径               | 主要参数                              | 实现说明                                             |
|--------------------|-----------------------------------|--------------------------------------------------|
| /preUploadVideo    | fileName、chunks                   | 创建上传任务，将文件名、分片数和临时路径等信息写入 Redis，并返回 uploadId。    |
| /uploadVideo       | chunkFile、<br>chunkIndex、uploadId | 校验上传任务、文件大小和分片序号后，将当前分片保存到临时目录，并更新 Redis 中的上传进度。 |
| /deleteUploadVideo | uploadId                          | 删除 Redis 中的上传任务，并清理对应临时上传目录。                     |

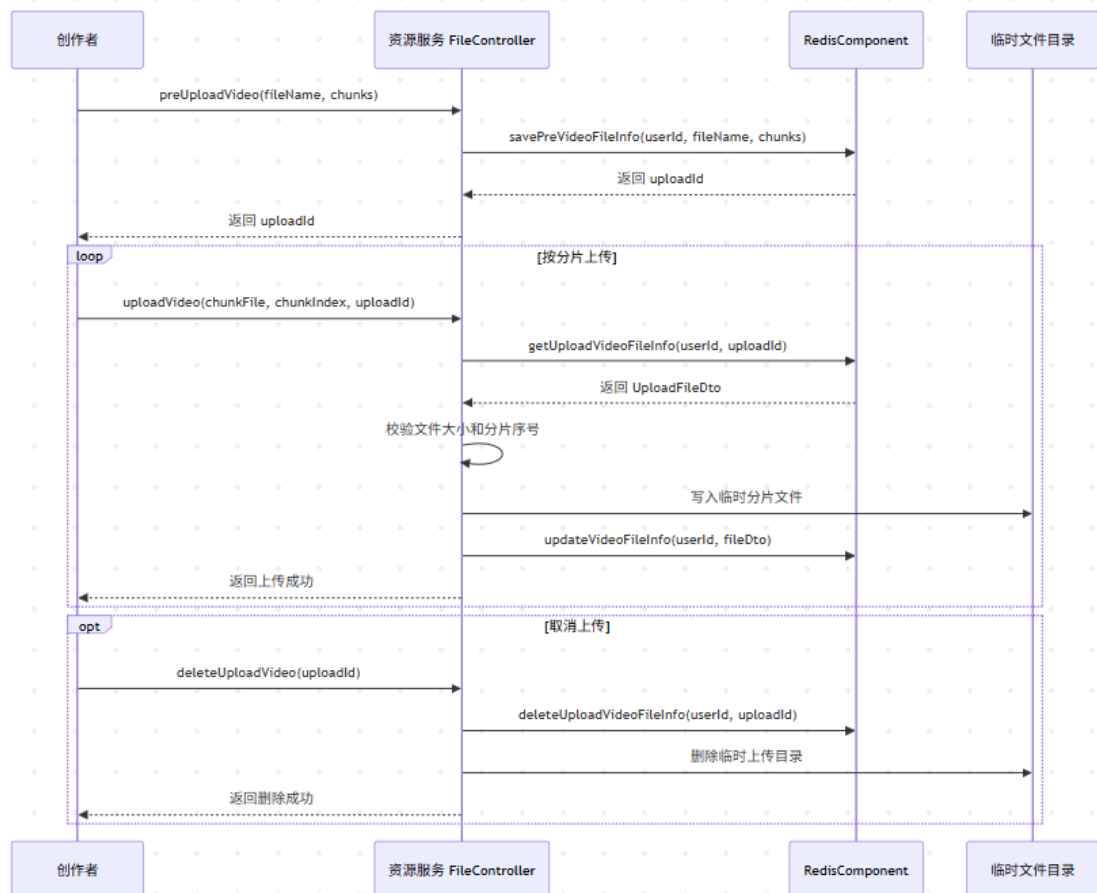


图 5-6 视频分片上传时序图

视频分片上传过程如图 5-6 所示。该图只描述资源上传阶段，不继续展开文件合并和转码处理，避免与后续“视频转码与资源服务实现”部分重复。

上传阶段产生的 `uploadId` 并不等同于视频编号。它只表示一次临时文件上传任务，真正的视频编号和分 P 文件编号要在投稿保存阶段生成。这样可以把“文件是否上传完成”和“投稿信息是否保存”分开处理，避免用户只上传了临时文件但没有提交投稿时，系统直接生成业务视频记录。

### 5.2.3 投稿信息保存与状态初始化实现

投稿保存入口为 `/ucenter/postVideo`。接口接收视频编号、封面、标题、分类、投稿类型、标签、简介、互动配置和 `uploadFileList` 等参数，其中 `uploadFileList` 会被反序列化为 `VideoInfoFilePost` 列表。控制器设置当前用户编号后，调用 `saveVideoInfo` 完成投稿保存。

投稿保存的关键不是简单写入一条视频记录，而是根据新增投稿和编辑投稿两种情况分别处理。若请求中没有 `videoId`，系统生成新的视频编号，将投稿主记录初始化为转码中状态，并插入 `VideoInfoPost`。若请求中存在 `videoId`，系统先查询原有投稿数据，并拒绝处于转码中或待审核状态的视频继续编辑，避免异步转码或审核结果覆盖新的投稿内容。

投稿信息保存与状态初始化过程如图 5-7 所示。



图 5-7 投稿信息保存与状态初始化流程图

编辑投稿时，系统会读取数据库中已有的分 P 文件列表，并与本次提交的 `uploadFileList` 进行比较。若数据库中的某个分 P 文件在本次提交列表中不存在，说明该分 P 被删除，系统会删除对应的投稿态文件记录，并将文件路径写入 Redis 删除队列；若本次提交中存在 `fileId` 为空的分 P，说明它是新增文件，系统会生成新的文件编号，并将转码结果初始化为转码中。若没有新增文件，但标题、封面、标签、简介、投稿类型或分 P 文件名发生变化，则视频状态会被更新为待审核。

所有分 P 文件处理完成后，系统调用批量新增或更新方法保存 `VideoInfoFilePost`

列表。对于新增的分P文件，系统还会补充用户编号和视频编号，并调用 `addFile2TransferQueue` 写入转码队列。资源服务后续从该队列中取出文件任务，完成临时分片合并和播放资源生成。通过这种方式，投稿接口只负责业务数据保存和状态初始化，不等待耗时的视频转码处理完成。

### 5.3 视频转码与资源服务实现

视频转码与资源服务实现主要处理投稿视频文件从临时分片资源到可播放资源的转换过程。投稿保存阶段已经将新增分P文件写入 Redis 转码队列。

资源服务并不通过同步接口等待视频处理完成，而是由后台任务持续消费 Redis 中的转码任务。每个转码任务对应一个投稿态分P文件，任务对象中包含用户编号、视频编号、文件编号和上传标识等信息。资源服务根据这些字段定位上传阶段产生的临时目录，并在处理完成后通过主站服务回写文件转码结果。

#### 5.3.1 资源服务职责

资源服务的职责主要集中在文件资源本身，不直接决定视频是否可以发布。它负责从 Redis 转码队列中读取待处理分P文件，完成临时目录迁移、分片合并、视频信息读取、编码兼容处理和 HLS 播放资源生成，并将处理结果回写给主站业务服务。播放阶段，资源服务还负责根据分P文件编号读取 m3u8 索引文件和 ts 切片文件。

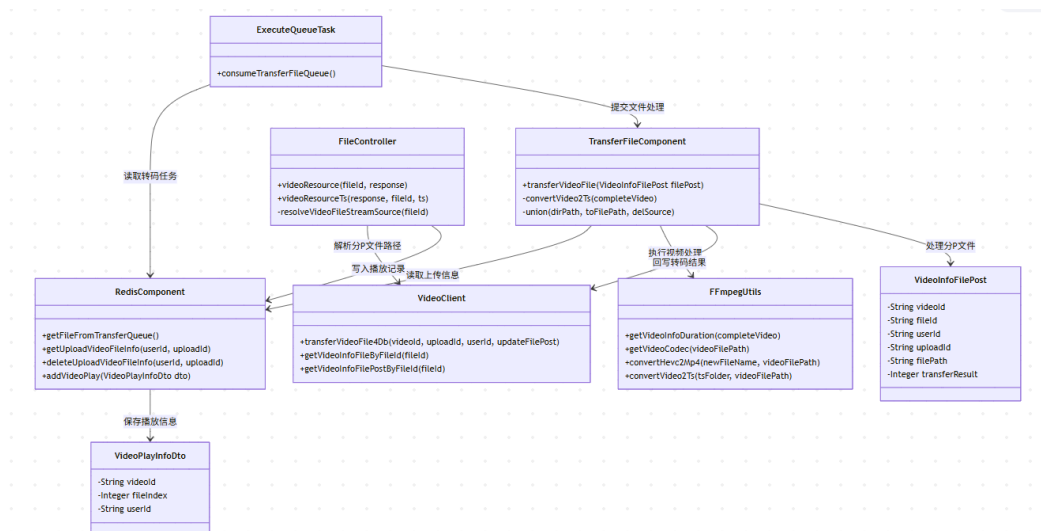


图 5-8 资源服务关键类图

资源服务关键类之间的关系如图 5-8 所示。ExecuteQueueTask 负责从 Redis 中获取转码任务，TransferFileComponent 负责实际文件处理，FFmpegUtils 负责执行视频处理命令，FileController 负责播放资源输出，VideoClient 则用于资源服务与主站视频数据之间的交互。

资源服务中的关键类和职责如表 5-5 所示。

表 5-5 资源服务关键类说明

| 类名                    | 核心方法                                                       | 实现说明                                                         |
|-----------------------|------------------------------------------------------------|--------------------------------------------------------------|
| ExecuteQueueTask      | consumeTransferFileQueue                                   | 后台循环读取 Redis 转码队列，取出待处理的 VideoInfoFilePost 后交给转码组件处理。        |
| TransferFileComponent | transferVideoFile                                          | 根据上传标识读取临时文件信息，完成目录迁移、分片合并、视频时长读取、转码和结果回写。                   |
| FFmpegUtils           | getVideoInfoDuration、<br>getVideoCodec、<br>convertVideo2Ts | 封装 FFmpeg 和 FFprobe 命令，用于读取视频时长、获取编码格式、处理 HEVC 编码和生成 HLS 资源。 |
| FileController        | videoResource、<br>videoResourceTs                          | 根据分 P 文件编号读取 m3u8 和 ts 文件，并在正式视频访问时写入播放记录。                   |
| VideoClient           | transferVideoFile4Db、<br>getVideoInfoFileById              | 作为资源服务访问主站视频数据的客户端，用于回写转码结果和查询分 P 文件路径。                      |

通过上述划分，资源服务内部形成了较为清晰的处理边界。队列任务只负责触发处理，转码组件负责文件转换，工具类负责执行 FFmpeg 命令，控制器负责播放资源读取，主站服务负责保存最终业务状态。

### 5.3.2 视频文件合并与转码实现

视频文件合并与转码由 TransferFileComponent.transferVideoFile 完成。资源服务从队列中取得分 P 文件任务后，首先根据用户编号和 uploadId 从 Redis 中读取上传任务信息，并定位临时上传目录。随后，系统将临时目录复制到正式视频资源目录，删除临时目录和 Redis 中的上传任务信息，避免临时文件长期占用空间。

分片合并时，系统按照分片序号依次读取临时目录中的分片文件，并写入完整视频文件。合并完成后，系统通过 FFprobe 读取视频时长，并记录完整文件大小和正

式资源路径。如果处理过程中出现目录不存在、分片缺失或命令执行异常，系统会将该分 P 文件的转码结果设置为失败，并在最后统一回写主站服务。

在编码处理方面，系统会先调用 `FFmpegUtils.getVideoCodec` 读取视频编码格式。若检测到视频编码为 HEVC，则先将其转换为 H.264 编码，再继续生成 HLS 播放资源；若视频编码已经符合处理要求，则直接进入 HLS 生成步骤。这样处理主要是为了减少部分浏览器或播放环境对 HEVC 编码支持不足的问题。

无论转码成功还是失败，资源服务最终都会调用 `VideoClient.transferVideoFile4Db` 将处理结果回写给主站服务。成功时回写视频时长、文件大小、资源路径和成功状态；失败时回写失败状态。主站服务再根据同一视频下所有分 P 文件的处理结果，决定投稿视频后续是否进入待审核状态。

视频文件合并与转码实现过程如算法 2 所示。

---

**算法 2** 视频文件合并与转码实现算法

---

输入： Redis 转码队列任务  $T$

输出： 分 P 文件转码结果  $R$

```
1: 从 Redis 转码队列读取分 P 文件任务  $T$ 
2: if  $T$  为空 then
3:   等待一段时间后继续轮询转码队列
4:   return 空结果
5: end if
6: 初始化  $R.transferResult \leftarrow \text{FAIL}$ 
7: 根据  $T.userId$  和  $T.uploadId$  读取上传任务信息  $U$ 
8: 定位临时上传目录  $tempDir$  和正式资源目录  $targetDir$ 
9: 将  $tempDir$  复制到  $targetDir$ ，并清理临时目录和 Redis 上传状态
10: 按照分片序号依次读取  $targetDir$  中的分片文件
11: 将所有分片写入完整视频文件  $videoPath$ 
12: 使用 FFprobe 读取视频时长  $duration$ ，并统计文件大小  $fileSize$ 
13: 设置  $R.duration$ 、 $R.fileSize$  和  $R.filePath$ 
14: 调用 FFmpegUtils.getVideoCodec 获取视频编码  $codec$ 
15: if  $codec$  为 HEVC then
16:   使用 FFmpeg 将视频转换为 H.264 编码文件
17: end if
18: 使用 FFmpeg 生成 HLS 的 index.m3u8 和 ts 切片文件
19: 设置  $R.transferResult \leftarrow \text{SUCCESS}$ 
20: if 处理过程中出现目录缺失、分片缺失或命令执行异常 then
21:   保持  $R.transferResult \leftarrow \text{FAIL}$ 
22: end if
23: 调用 VideoClient.transferVideoFile4Db 回写  $R$ 
24: return  $R$ 
```

---



### 5.3.3 HLS 播放资源生成与访问实现

HLS 播放资源生成由 FFmpegUtils.convertVideo2Ts 完成。系统先将完整视频转换为中间 index.ts 文件,再按照固定时间切分为多个 ts 片段,并生成 index.m3u8 索引文件。项目中切片时间设置为 10 秒,播放器访问视频时先读取 m3u8 索引文件,再根据索引继续请求对应的 ts 切片。

播放资源访问由资源服务的 FileController 提供。播放页并不直接持有服务器文件真实路径,而是携带分 P 文件编号访问 /videoResource/{fileId}。资源服务根据 fileId 先查询正式分 P 文件;如果正式文件不存在,则继续查询投稿态分 P 文件,用于支持预览类访问。若两类数据都不存在,资源服务返回 404 状态。

HLS 播放资源访问过程如图 5-9 所示。

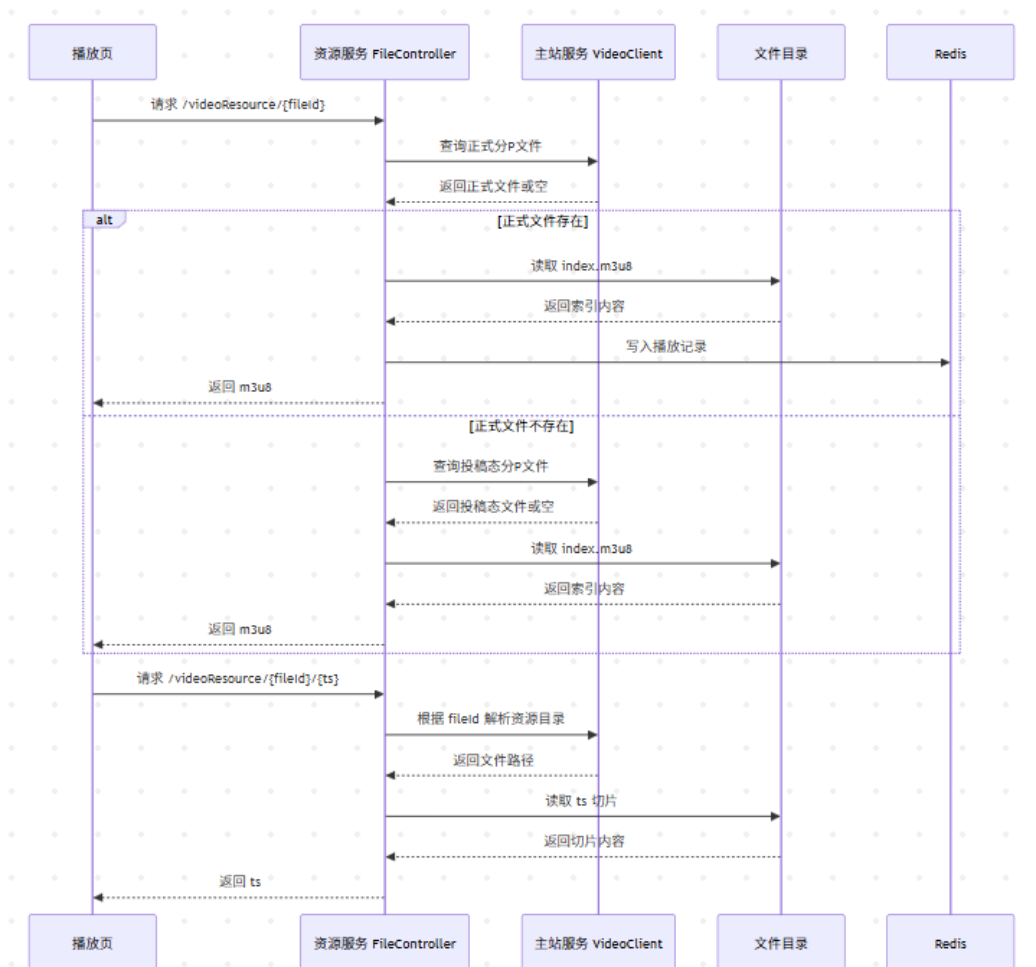


图 5-9 HLS 播放资源访问时序图

当请求的是 `/videoResource/{fileId}` 时,资源服务读取对应目录下的 `index.m3u8` 文件并写入响应。若该文件属于正式发布视频,系统还会构造 `VideoPlayInfoDto`, 保存视频编号、分 P 序号和可选的用户编号,并写入 Redis 播放记录队列。后续统计任务可以基于该队列异步更新播放量,避免每次播放请求都直接修改数据库。

当播放器继续请求 `/videoResource/{fileId}/{ts}` 时,资源服务根据同一个 `fileId` 解析资源目录,再读取对应的 `ts` 切片文件。该接口只负责切片文件读取,不重复写入播放记录。通过这种方式,系统既能够隐藏服务器真实文件路径,也能够将播放统计控制在索引文件访问阶段,减少切片频繁请求带来的额外统计开销。

## 5.4 互动功能模块实现

互动功能模块主要由互动服务承担,代码中对应评论、弹幕和用户行为三类接口。评论和弹幕属于内容型互动,需要保存用户提交的文本内容以及所属视频或分 P 文件;点赞、收藏、投币和评论赞踩属于行为型互动,主要通过用户编号、视频编号、评论编号和行为类型记录用户操作。由于互动功能在系统中属于基础业务支撑,本节不再展开完整业务流程,而是从接口、数据对象和统计字段更新角度进行说明。

互动模块中的主要接口如表 5-6 所示。其中,评论和弹幕的读取接口允许未登录用户访问,但发送评论、发送弹幕以及执行用户行为操作需要登录校验。系统在写入互动数据前,会先根据视频编号读取视频信息,并判断视频是否存在以及评论区或弹幕功能是否被关闭。

表 5-6 互动功能主要接口说明

| 接口路径                              | 主要参数                                                                                               | 实现说明                              |
|-----------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------|
| <code>/comment/postComment</code> | <code>videoId</code> 、 <code>content</code> 、 <code>replyCommentId</code>                          | 保存评论或回复评论,写入评论表,并在顶级评论新增时更新视频评论数。 |
| <code>/comment/loadComment</code> | <code>videoId</code> 、 <code>pageNo</code> 、 <code>orderType</code>                                | 根据视频编号分页查询评论,并在用户已登录时补充评论点赞或点踩状态。 |
| <code>/danmu/postDanmu</code>     | <code>videoId</code> 、 <code>fileId</code> 、 <code>text</code> 、 <code>time</code>                 | 保存弹幕内容,记录分 P 文件编号和播放时间,并更新视频弹幕数。  |
| <code>/danmu/loadDanmu</code>     | <code>videoId</code> 、 <code>fileId</code>                                                         | 按分 P 文件编号加载弹幕列表,并按照弹幕编号升序返回。      |
| <code>/userAction/doAction</code> | <code>videoId</code> 、 <code>actionType</code> 、 <code>actionCount</code> 、 <code>commentId</code> | 统一处理视频点赞、收藏、投币以及评论赞踩等用户行为。        |

互动模块中的关键类关系如图 5-10 所示。控制器层负责接收页面请求并解析当前用户信息，服务层负责完成业务校验、行为写入和统计字段更新，数据对象则分别对应评论、弹幕和用户行为记录。

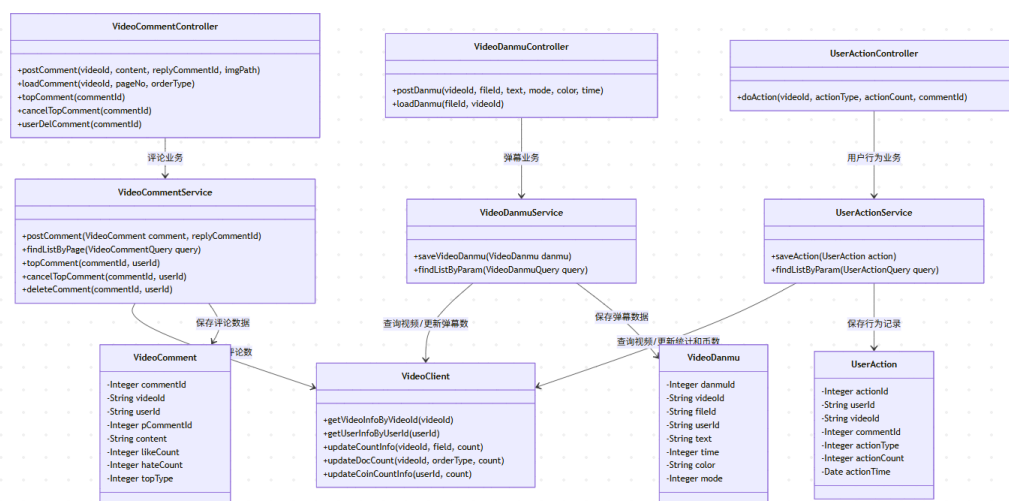


图 5-10 互动功能关键类图

#### 5.4.1 评论与弹幕实现

评论功能由 VideoCommentController 和 VideoCommentService 实现。用户发表评论时，接口会接收视频编号、评论内容、回复评论编号和可选图片路径，并从登录信息中补充用户编号、昵称和头像。服务层首先根据视频编号查询视频信息，如果视频不存在或评论区已关闭，则直接返回异常；如果是回复评论，则继续查询被回复评论，确定父评论编号、被回复用户编号和被回复用户昵称等信息。评论写入成功后，若该评论为顶级评论，系统会调用主站服务更新视频评论数。

评论读取接口根据视频编号构造 VideoCommentQuery，并支持按点赞数或评论时间排序。查询时系统可以加载一级评论及其子评论，用于播放页评论区展示。若当前用户已登录，系统还会根据用户编号和视频编号查询评论点赞、点踩记录，并将这些行为状态与评论分页结果一起返回。这样页面可以直接展示用户是否已经对某条评论执行过点赞或点踩操作。

弹幕功能由 VideoDanmuController 和 VideoDanmuService 实现。发送弹幕时，系统保存视频编号、分 P 文件编号、弹幕内容、显示模式、颜色、播放时间和用户编

号。与评论不同，弹幕必须绑定具体的 `fileId` 和播放时间，因此多分 P 视频可以分别加载各自的弹幕数据。服务层在保存弹幕前同样会查询视频信息，并判断视频是否关闭弹幕功能。弹幕保存成功后，系统更新视频弹幕数和搜索文档中的弹幕统计字段。

评论与弹幕处理过程如算法 3 所示。

---

**算法 3** 评论与弹幕处理实现算法

---

输入： 互动请求 *request*，当前登录用户 *user*

输出： 互动处理结果 *result*

```
1: if user 为空 then
2:   返回未登录异常，不写入互动数据
3: end if
4: 根据 request.videoId 查询视频信息 video
5: if video 不存在 then
6:   返回业务异常
7: end if
8: if request.type 为评论 then
9:   if video.interaction 表示评论区关闭 then
10:    返回评论区关闭异常
11:  end if
12:   构造评论对象 comment，写入视频编号、用户编号、昵称、头像和评论内容
13:   if request.replyCommentId 不为空 then
14:     查询被回复评论 parent
15:     根据 parent 确定一级评论编号、被回复用户编号和被回复用户信息
16:   else
17:     设置 comment.pCommentId  $\leftarrow 0$ 
18:   end if
19:   写入评论记录
20:   if comment.pCommentId = 0 then
21:     调用主站服务增加视频评论数
22:   end if
23: else if request.type 为弹幕 then
24:   if video.interaction 表示弹幕关闭 then
25:     返回弹幕关闭异常
26:   end if
27:   构造弹幕对象 danmu
28:   写入视频编号、分 P 文件编号、弹幕内容、颜色、模式、播放时间和用户编号
29:   写入弹幕记录
30:   调用主站服务增加视频弹幕数
31:   调用搜索组件同步更新搜索文档中的弹幕统计字段
32: end if
33: 返回处理成功结果 result
```

---

效果如图 5-11 和图 5-12 所示。图 5-11 展示了视频播放页中的弹幕功能，用户

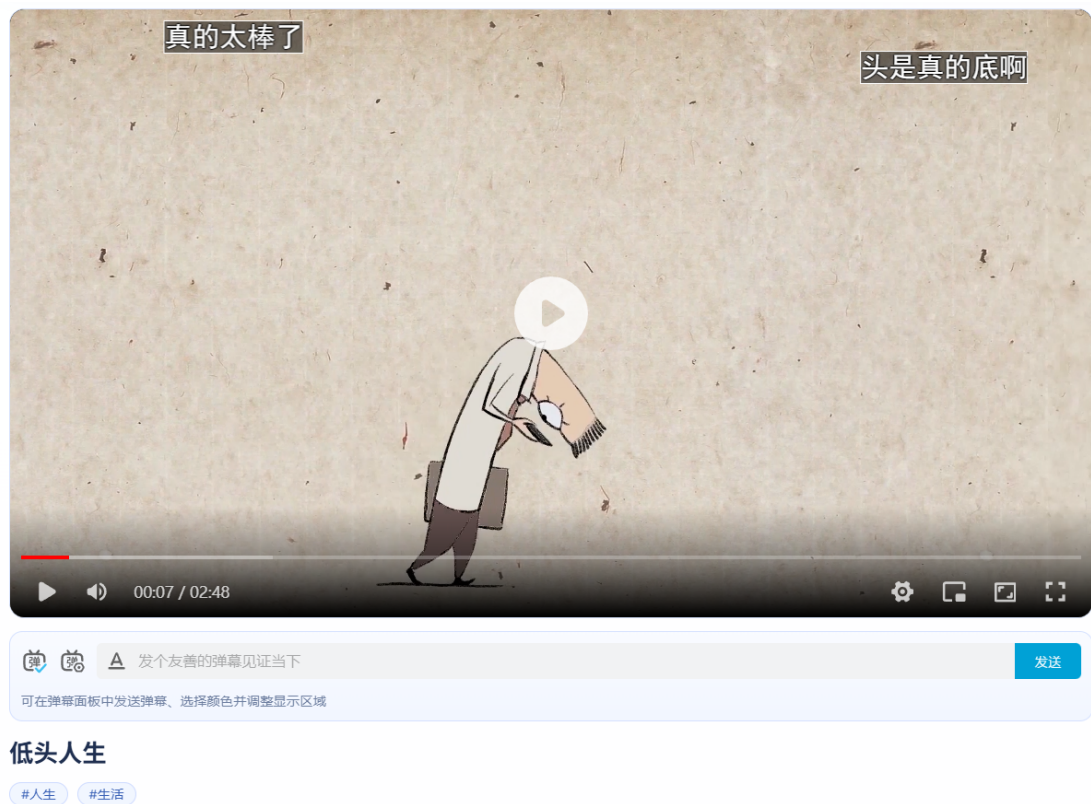


图 5-11 视频播放页弹幕展示界面

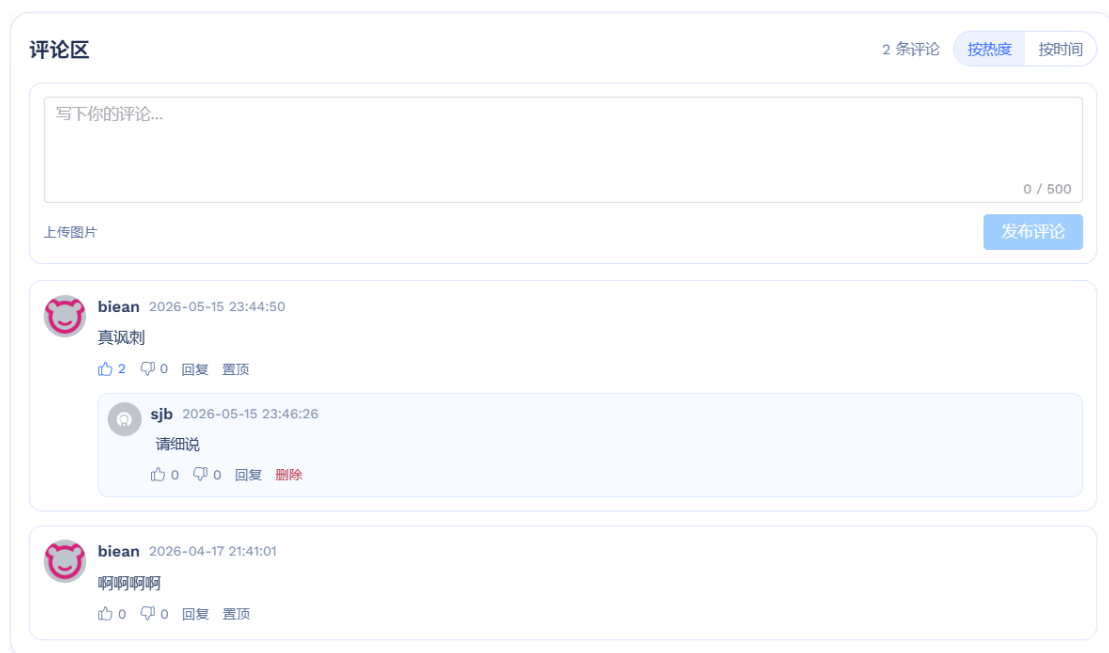


图 5-12 视频播放页评论区展示界面

可以在播放器下方输入弹幕内容，选择弹幕显示方式和颜色后发送，已发送的弹幕会按照播放时间叠加显示在视频画面上。图 5-12 展示了评论区功能，页面支持发布评论、上传评论图片、按热度或时间排序，并提供评论点赞、点踩、回复、置顶和删除等交互操作。上述界面分别对应算法中的弹幕处理分支和评论处理分支，说明系统能够将互动接口处理结果渲染为可直接操作的播放页互动功能。

### 5.4.2 点赞、收藏、投币与关注实现

点赞、收藏和投币统一由 `/userAction/doAction` 接口处理。接口接收视频编号、行为类型、行为数量和评论编号等参数，并在登录校验通过后构造 `UserAction` 对象。服务层首先根据视频编号查询视频信息，再根据 `actionType` 判断具体行为类型。如果行为类型不存在，系统会直接返回业务异常，避免非法行为类型写入数据库。

视频点赞和视频收藏属于可取消行为。系统会根据视频编号、评论编号、行为类型和用户编号查询已有行为记录：若记录已经存在，则删除原行为并将统计字段减一；若记录不存在，则新增行为并将统计字段加一。对于收藏行为，系统还会同步更新搜索文档中的收藏统计，使搜索排序或热度展示能够反映收藏变化。

投币行为与点赞、收藏不同，通常不作为可取消行为处理。系统会先判断用户是否给自己的视频投币，若是则返回异常；随后判断是否已经投过币，避免重复投币。投币前还需要扣减当前用户的币数，并给视频作者增加相应币数。只有这些操作成功后，系统才写入投币行为记录，并更新视频投币统计。该部分使用全局事务控制，主要是为了保证行为记录、用户币数和视频统计之间保持一致。

从当前代码中的行为枚举看，已实现的用户行为主要包括视频点赞、视频收藏、视频投币、评论点赞和评论点踩等。关注功能如果后续加入，可以继续沿用类似的用户关系记录方式，即以当前用户编号和目标用户编号作为唯一关系进行新增或取消；但在本文实现说明中，重点仍放在已有的 `UserAction` 行为处理逻辑上。

### 5.4.3 互动统计数据更新实现

互动统计数据主要由互动服务在写入行为时同步更新。评论新增时，系统只在顶级评论写入后增加视频评论数；回复评论属于评论内部层级变化，不单独增加视频维度的评论统计。删除顶级评论时，系统会减少视频评论数，并删除该评论下的二级评论，避免子评论失去父级关系。

弹幕新增后，系统会增加视频弹幕数，并更新搜索文档中的弹幕统计。弹幕读取时只根据分 P 文件编号加载，不会重复修改统计数据。这样可以区分“发送弹幕”与“读取弹幕”两类操作，避免播放页频繁加载弹幕时影响统计字段。

点赞和收藏通过行为记录是否存在来决定新增或取消，并同步更新视频表中的对应统计字段。评论点赞和点踩还存在互斥关系，用户点赞某条评论时，如果之前已经点踩，系统会先删除对立行为，再更新评论表中的点赞数和点踩数。投币则涉及用户币数扣减、作者币数增加、投币行为记录写入和视频投币数更新，因此比普通点赞和收藏需要更严格的一致性控制。

整体来看，互动统计更新并不是单独的统计任务，而是与评论、弹幕和用户行为写入同时完成。明细表用于记录用户具体做过什么操作，统计字段用于减少页面展示时的聚合查询成本。对于本项目规模而言，这种同步更新方式实现较为直接，也能够满足播放页、首页和搜索结果中的基础统计展示需要。

### 5.5 后台管理功能实现

后台管理功能主要用于处理管理员登录、分类维护、投稿内容查询和视频审核等操作。由于第三章和第四章已经对后台管理端的功能需求和服务划分进行了说明，本节不再重复描述后台管理端“能够做什么”，而是从管理端接口和关键业务类角度说明其实现方式。

在项目代码中，后台管理端主要由 `streama-cloud-admin` 模块提供接口，外部访问路径以 `/admin` 为前缀。管理员账号相关接口由 `AccountController` 处理，分类维护接口由 `CategoryController` 处理，视频内容和审核相关接口由 `VideoInfoController` 处理。其中，视频审核相关数据并不直接由后台服务独立维护，而是通过 `WebClient` 调用主站业务服务完成投稿视频查询、AI 审核结果读取和人工审核处理。

#### 5.5.1 后台管理端整体实现

后台管理端账号相关接口如表 5-7 所示。

主要包括管理员验证码生成、管理员登录、后台令牌保存和管理接口调用等内容。管理员登录前，系统通过 `/admin/account/checkCode` 生成验证码，并将验证码内容保存到 Redis 中。管理员提交账号、密码和验证码后，`AccountController.login` 会先校验验证码，再校验配置文件中的管理员账号和密码。校验通过后，系统生成

表 5-7 后台管理端账号接口说明

| 接口路径                     | 主要参数                                    | 实现说明                                              |
|--------------------------|-----------------------------------------|---------------------------------------------------|
| /admin/account/checkCode | 无                                       | 使用 Kaptcha 生成验证码图片，将验证码文本写入 Redis，并返回验证码图片和验证码标识。 |
| /admin/account/login     | account、password、checkCodeKey、checkCode | 校验验证码、管理员账号和密码，登录成功后生成后台令牌并保存到 Redis。             |
| /admin/account/logout    | 无                                       | 清理后台登录 Cookie，使管理员退出后台管理端。                        |

后台登录令牌，并将其写入 Cookie，同时在 Redis 中保存管理员登录信息。

后台管理端关键类关系如图 5-13 所示。该图只展示本节涉及的主要控制器、服务调用对象和数据对象，不再展开网关转发过程，避免与前文系统总体架构图重复。

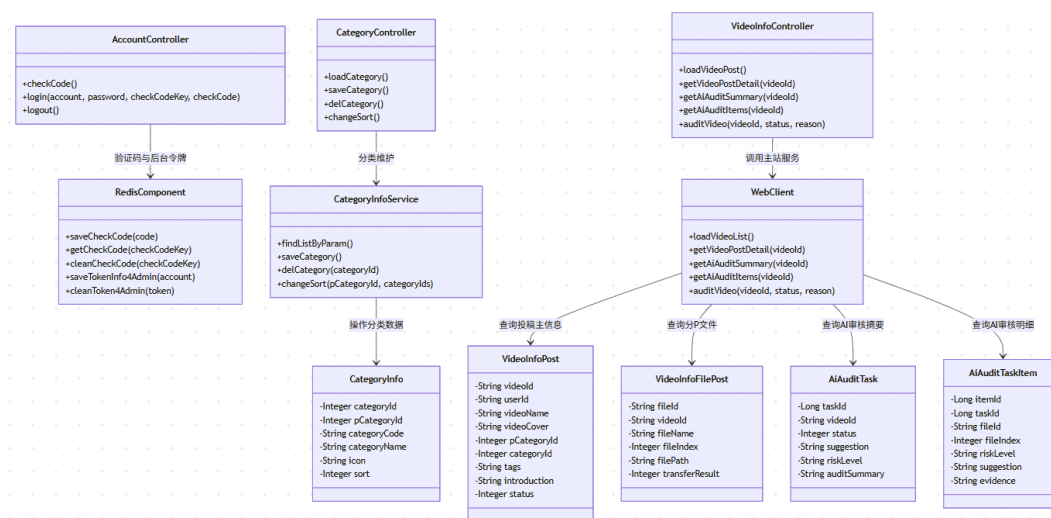


图 5-13 后台管理功能关键类图

从整体实现上看，后台管理端可以分为本地处理和远程调用两类逻辑。管理员登录和分类维护主要由后台管理服务内部完成；视频投稿列表、审核详情、AI 审核摘要和人工审核操作则通过 WebClient 转发给主站业务服务处理。这样可以避免后台服务直接维护投稿视频表和 AI 审核任务表，使视频审核状态仍然由主站业务流程统一管理。



### 5.5.2 分类与内容管理实现

分类管理由 `CategoryController` 提供接口，主要包括分类加载、分类保存、分类删除和分类排序调整。分类加载接口会构造 `CategoryInfoQuery` 查询对象，并设置排序条件为 `sort asc`，同时将查询结果转换为树形结构返回。这样后台页面可以按照一级分类和二级分类的层级关系展示分类数据。

分类维护相关接口如表 5-8 所示。

表 5-8 分类管理接口说明

| 接口路径 (admin)                        | 主要参数                                                                                                                                    | 实现说明                                                            |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| <code>/category/loadCategory</code> | <code>pCategoryId</code> 、<br><code>categoryNameFuzzy</code> 、<br><code>pageNo</code>                                                   | 查询分类数据，并按照排序字段组织为分类树结构。                                         |
| <code>/category/saveCategory</code> | <code>pCategoryId</code> 、<br><code>categoryId</code> 、<br><code>categoryCode</code> 、<br><code>categoryName</code> 、 <code>icon</code> | 新增或修改分类信息，由 <code>CategoryInfoService.saveCategory</code> 完成保存。 |
| <code>/category/delCategory</code>  | <code>categoryId</code>                                                                                                                 | 根据分类编号删除分类，由服务层完成删除前的业务校验。                                      |
| <code>/category/changeSort</code>   | <code>pCategoryId</code> 、<br><code>categoryIds</code>                                                                                  | 按照传入的分类编号顺序调整同级分类排序。                                            |

内容管理主要指后台对投稿视频列表的查询。后台接口 `WebClient.loadVideoList` 接收 `VideoInfoPostQuery` 查询对象，可以按照视频编号、标题、用户编号、分类、投稿状态、投稿类型和时间范围等条件查询投稿态视频数据。该接口本身不直接处理审核逻辑，而是用于为后台列表页提供分页数据。

从实现角度看，分类管理是后台服务本地业务，内容管理则通过 `WebClient.loadVideoList` 调用主站服务查询投稿数据。这样分类维护和视频投稿查询在代码中保持了比较清晰的边界，也避免后台管理模块重复实现视频投稿查询逻辑。

### 5.5.3 视频审核管理实现

视频审核管理是后台管理功能中相对重要的部分。后台视频审核入口位于 `VideoInfoController`，该控制器并不直接操作投稿视频表，而是通过 `WebClient` 调用主站业务服务完成审核相关处理。后台页面加载审核列表时，调用 `WebClient.loadVideoList` 查询投稿态视频；查看某个视频详情时，调用 `WebClient.getVideoPostDe`

tail 获取投稿主信息和分 P 文件信息；查看 AI 审核结果时，则分别调用 `WebClient.getAiAuditSummary` 和 `WebClient.getAiAuditItems` 获取视频级审核摘要和分 P 级审核明细。

视频审核相关接口如表 5-9 所示。

表 5-9 视频审核管理接口说明

| 接口路径 (admin)                               | 主要参数                                                                                          | 实现说明                                 |
|--------------------------------------------|-----------------------------------------------------------------------------------------------|--------------------------------------|
| <code>/videoInfo/loadVideoList</code>      | <code>status</code> 、 <code>videoNameFuzzy</code> 、 <code>userId</code> 、 <code>pageNo</code> | 查询投稿态视频分页列表，用于后台审核列表展示。              |
| <code>/videoInfo/getVideoPostDetail</code> | <code>videoId</code>                                                                          | 读取投稿视频主信息和分 P 文件信息，用于审核详情页展示。        |
| <code>/videoInfo/getAiAuditSummary</code>  | <code>videoId</code>                                                                          | 查询视频级 AI 审核任务摘要，包括审核建议、风险等级和任务状态等信息。 |
| <code>/videoInfo/getAiAuditItems</code>    | <code>videoId</code>                                                                          | 查询分 P 级 AI 审核明细，用于展示风险片段和审核依据。       |
| <code>/videoInfo/auditVideo</code>         | <code>videoId</code> 、 <code>status</code> 、 <code>reason</code>                              | 提交人工审核结果。审核通过时进入发布同步逻辑，审核不通过时保存驳回原因。 |

管理员执行审核操作时，接口会接收视频编号、审核状态和驳回原因等参数。若审核状态为通过，主站服务会继续完成投稿态数据到正式发布数据的同步；若审核状态为不通过，系统只更新投稿态视频的状态和驳回原因，不会写入正式视频表。由于 AI 审核任务编排和发布同步流程在后续章节中单独说明，本节只保留后台审核入口和数据读取方式，不再重复展开完整审核链路。

通过上述实现，后台管理端在代码上承担的是管理入口和审核操作入口的作用。分类管理用于维护视频分区结构，内容管理用于查看投稿视频数据，视频审核管理用于读取投稿详情、AI 审核结果并提交人工审核结论。真正的视频状态更新、AI 审核结果保存和发布数据同步仍由主站业务服务负责处理。

## 5.6 AI 审核任务编排与人机协同实现

在 AI 审核这个环节当中，任务编排主要负责去衔接视频转码完成之后的自动初审以及后台的人工终审。鉴于前文已经对消息队列、审核服务以及后台管理端三者

之间的总体协作关系进行了说明，所以本节不再去重复绘制完整的消息时序图，而是把重点放在说明 AI 审核任务如何去落库、审核结果如何去进行更新，以及人工审核通过之后如何去同步正式发布的数据。

在项目实现中，AI 审核相关数据并不直接写入投稿视频主表，而是拆分为视频级任务和分 P 级明细两类数据。AiAuditTask 用于保存一次视频级 AI 审核任务，包括任务状态、AI 建议、风险等级和审核摘要等信息；AiAuditTaskItem 用于保存每个分 P 文件的审核结果，包括分 P 建议、风险分值、风险标签和命中片段等信息。后台审核页面读取这些数据后，管理员再结合视频内容作出最终通过或驳回决定。

AI 审核与发布同步涉及的关键对象如图 5-14 所示。该图只保留任务编排、审核结果和发布同步相关的核心类，不再展示查询条件对象和分页封装对象，以减少与前面章节的重复。

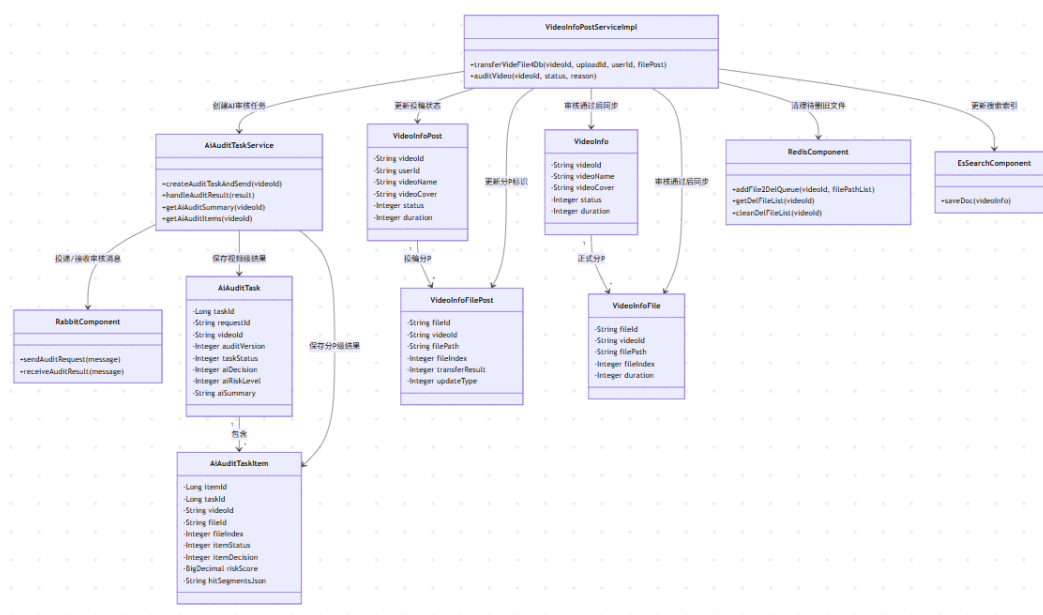


图 5-14 AI 审核任务与发布同步关键类图

从状态角度看，视频投稿、AI 任务和人工审核之间存在较明确的先后关系。投稿视频先经过资源服务转码，转码完成后进入待审核状态；AI 审核任务用于生成初审建议；人工审核通过后，系统才会将投稿态数据同步到正式发布表。该状态变化如图 5-15 所示。

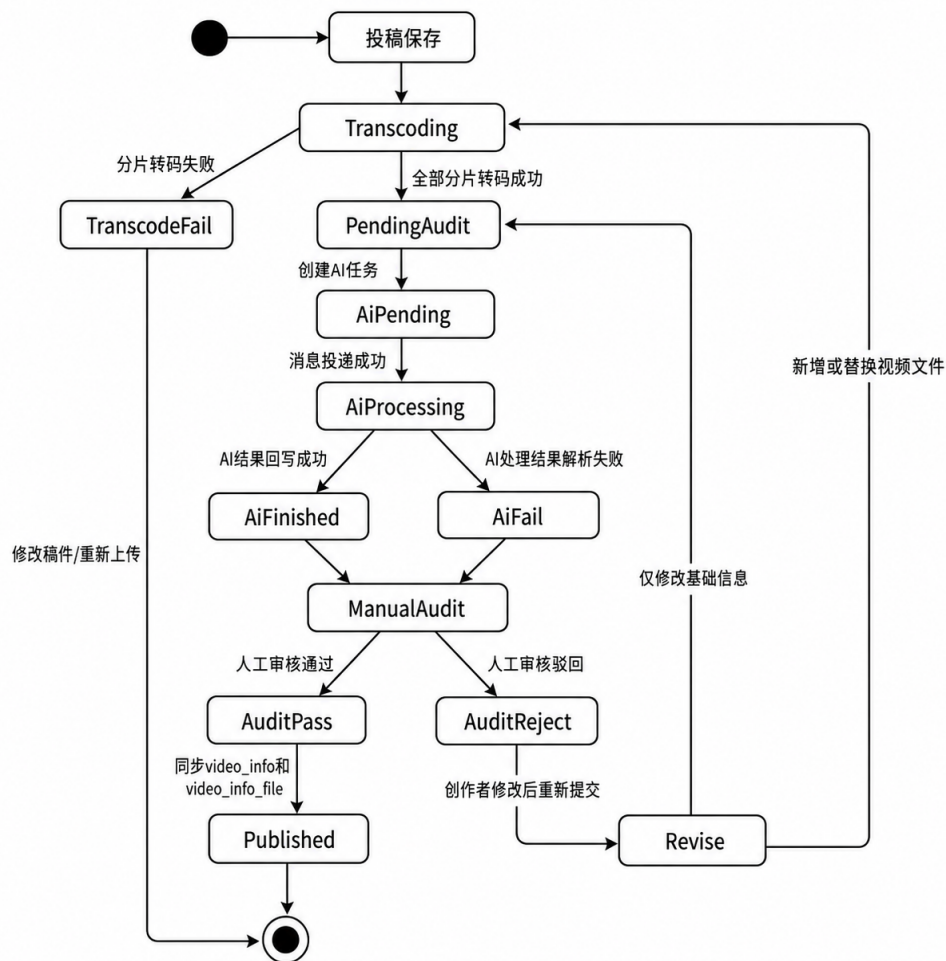


图 5-15 AI 审核与发布状态流转图

### 5.6.1 AI 审核任务创建与消息投递实现

AI 审核任务创建发生在视频分 P 文件转码完成之后。资源服务完成文件处理后，会将转码结果回写给主站服务。主站服务根据同一视频下所有分 P 文件的转码状态进行判断：如果存在转码失败的分 P 文件，则视频状态更新为转码失败；如果所有分 P 文件均已转码成功，则视频进入待审核状态，并具备创建 AI 审核任务的条件。

任务创建时，系统先生成一条视频级任务记录，对应 `ai_audit_task` 表。该记录用于保存视频编号、请求编号、审核版本、任务状态、触发时间、模型信息和后续 AI 返回的视频级结论。随后，系统根据该视频下的分 P 文件生成多条任务明细记录，对应 `ai_audit_task_item` 表。每条明细记录保存分 P 文件编号、文件序号、文件名、文件路径、时长和初始审核状态等信息。

AI 审核任务的主要数据对象如表 5-10 所示。

表 5-10 AI 审核任务主要数据对象说明

| 数据对象              | 关键字段                                                | 实现作用                                   |
|-------------------|-----------------------------------------------------|----------------------------------------|
| AiAuditTask       | videoId、requestId、taskStatus、aiDecision、aiRiskLevel | 保存一次视频级 AI 审核任务，用于后台列表展示 AI 建议和整体风险摘要。 |
| AiAuditTaskItem   | taskId、fileId、fileIndex、itemDecision、riskScore      | 保存每个分 P 文件的 AI 审核明细，用于审核详情页定位具体风险片段。   |
| VideoInfoPost     | videoId、status、duration                             | 保存投稿态视频主信息，转码完成后进入待审核状态。               |
| VideoInfoFilePost | fileId、videoId、filePath、transferResult              | 保存投稿态分 P 文件信息，为 AI 审核提供文件路径和分 P 信息。    |

消息投递时，系统不直接把视频文件写入消息队列，而是将审核任务编号、视频编号、分 P 文件编号和文件路径等信息封装为审核请求消息。AI 服务消费消息后，根据文件路径读取待审核视频，并在处理完成后返回结构化审核结果。通过“任务先落库，消息后投递”的方式，即使消息发送失败或 AI 服务处理异常，系统也能根据任务表保留任务状态和失败原因。

### 5.6.2 审核结果接收与任务状态更新实现

AI 服务完成审核后，会将视频级结果和分 P 级结果返回主站服务。主站服务接收结果时，首先根据请求编号或任务编号定位 AiAuditTask，再更新任务主表和任务明细表。视频级结果主要写入 AI 建议、风险等级、审核摘要、模型名称、模型版本和完成时间；分 P 级结果主要写入分 P 建议、风险分值、风险标签、命中片段和审核说明。

审核结果更新时需要注意任务状态的一致性。任务创建后通常处于待处理状态，消息投递或 AI 服务开始处理后进入处理中状态，AI 结果成功回写后更新为已完成状态。如果消息投递失败、AI 服务异常或结果解析失败，则任务应更新为失败状态，并记录错误信息。对于没有有效待审文件的情况，任务可以被标记为取消状态。

命中片段是 AI 审核结果中比较重要的部分。它通常以 JSON 形式保存到 hitSegmentsJson 字段中，用于记录风险出现的时间范围、风险类型、文本摘要、声音事

件和模型判断理由等内容。后台审核详情页可以根据这些数据展示分 P 级风险证据，帮助管理员快速定位问题片段。

由于 AI 审核结果只承担初审提示作用，因此 AI 结果回写不会直接把 `video_info_post.status` 修改为审核通过或审核不通过。投稿视频仍然保持待审核状态，等待管理员在后台执行人工审核操作。这样可以避免模型误判直接影响视频是否公开发布。

### 5.6.3 AI 初审与人工终审协同实现

AI 初审与人工终审的协同关系主要体现在决策边界上。AI 服务负责根据视频内容生成审核建议、风险等级和命中片段，后台管理端负责展示这些结构化结果，管理员负责作出最终审核结论。也就是说，AI 结果用于缩小人工复核范围，但不直接替代人工审核。

后台审核列表可以展示视频基础信息、投稿状态、AI 建议、风险等级和审核摘要。管理员进入审核详情后，可以继续查看分 P 文件列表和 AI 审核明细。如果某个分 P 文件存在风险标签或命中片段，管理员可以根据片段时间范围、风险说明和播放资源进行复核。对于 AI 建议通过的视频，管理员仍然可以人工驳回；对于 AI 建议驳回或人工复核的视频，管理员也可以结合实际内容进行判断。

该协同方式能够保证系统审核流程具有一定的自动化能力，同时仍然保留人工终审的控制权。AI 审核负责提高风险发现效率，人工审核负责保证最终发布结果符合平台规则。对于本文项目规模而言，这种处理方式实现成本较低，也更适合毕业设计系统的可展示和可解释要求。

### 5.6.4 审核通过与发布数据同步实现

审核通过与发布数据同步由主站服务中的人工审核逻辑完成。管理员在后台提交审核结果后，系统首先校验视频是否仍处于待审核状态，避免已经处理过的视频被重复审核。校验通过后，系统更新 `video_info_post.status`。如果审核结果为驳回，则只保存投稿态审核状态，不会写入正式视频表。

如果审核结果为通过，系统会将投稿态数据同步到正式发布数据中。主站服务先将 `VideoInfoPost` 复制为 `VideoInfo`，写入或更新正式视频主表；随后删除该视频原有的正式分 P 文件记录，再将 `VideoInfoFilePost` 列表复制为 `VideoInfoFile` 并批量写入正式分 P 文件表。同步完成后，视频才会进入首页、搜索和播放页等公

开视频访问流程。

审核通过时，系统还会将投稿态分 P 文件的更新标识重置为未更新状态，并清理 Redis 中等待删除的旧文件路径。如果该视频是首次发布，系统还会给投稿用户增加一定数量的奖励硬币。最后，系统调用 Elasticsearch 组件保存或更新视频索引，使审核通过的视频能够被关键词搜索命中。

审核不通过时，系统不会同步正式表，也不会更新搜索索引。创作者后续可以在创作中心看到稿件未通过状态，并根据审核意见修改后重新提交。通过这种投稿态表和正式发布表分离的方式，系统能够保证未审核通过的视频不会进入用户端公开访问链路。

### 5.7 多模态 AI 审核服务实现

多模态 AI 审核服务是 AI 初审的具体执行模块。该服务接收主站服务投递的视频审核请求后，按照分 P 文件逐个处理视频内容，并输出分 P 级和视频级审核结果。

#### 5.7.1 AI 审核服务结构与模型选型依据

AI 审核服务可以按照处理职责划分为审核编排层、模型能力层、片段构建层和结果生成层。审核编排层负责控制完整审核流程，包括接收审核请求、初始化模型服务、调用视频处理方法、收集异常信息和返回最终结果。模型能力层负责封装 VAD、Whisper、YAMNet 和 Qwen 等模型能力，其中当前多模态视觉语言模型使用 Qwen/Qwen3-VL-4B-Instruct。片段构建层负责将不同模态的数据按照时间戳对齐，生成可供模型审核的 segment 和审核窗口。结果生成层则负责将片段级结果归并为分 P 级和视频级审核建议。

AI 审核服务结构如图 5-16 所示。

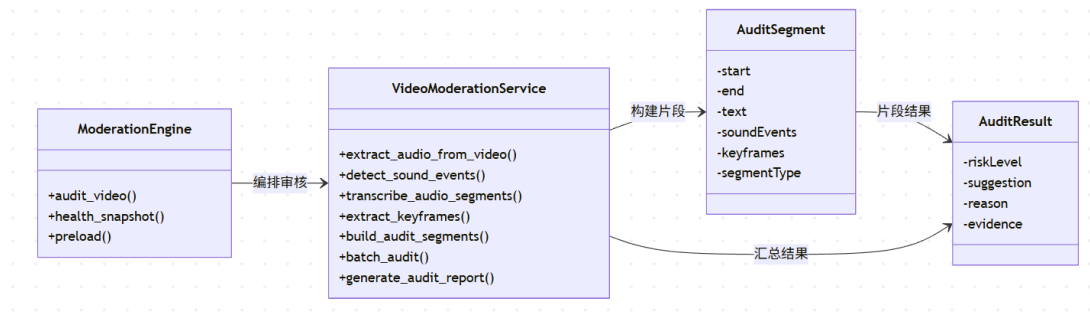


图 5-16 AI 审核服务结构图

审核编排层负责把控处理顺序，模型能力层负责完成具体分析，片段构建层负责组织多模态证据，结果生成层负责生成可落库、可展示和可复核的审核结果。

在审核方式选择上，本文参考了视频毒性内容检测相关研究。Maity 等提出的 ToxVidLM 面向视频内容中的毒性检测任务，将视频中的多模态信息引入检测过程，并通过编码模块、跨模态同步模块和多任务模块完成毒性、情感和严重程度等判断<sup>[34]</sup>。该研究说明，视频审核类任务需要结合语言内容、视觉画面及其上下文关系进行综合判断。结合本文系统实现，语音转写结果、声音事件、关键帧和投稿元数据被统一组织到审核窗口中，再由多模态模型生成片段级审核建议。

在具体模型选择上，本文主要结合任务适配性、输出可结构化程度和工程部署成本进行取舍。语音内容处理环节选择 Whisper，而不是直接使用通用文本生成模型，是因为该环节的输入是原始音频，核心任务是语音识别而不是文本生成。Whisper 面向语音识别任务设计，并基于大规模弱监督语音数据训练，在多语言、多任务语音识别场景中具有较好的泛化能力<sup>[4]</sup>。因此，Whisper 更适合将视频中的口播和对白转换为带时间信息的文本证据，供后续审核窗口使用。相比之下，通用文本生成模型主要处理已经给定的文本输入，不能直接完成音频转写，也难以提供与视频时间轴对应的语音片段结果。

声音事件识别环节选择 YAMNet，YAMNet 是面向声音事件分类的模型，能够预测 AudioSet 体系中的 521 类声音事件，并采用 MobileNet v1 结构，模型相对轻量，适合在预处理阶段对音频进行快速分类<sup>[35]</sup>。因此，本文没有仅依赖 Whisper 的转写结果，而是使用 YAMNet 补充非语音声音事件，从而提高对无对白或低语音视频的覆盖能力。

多模态综合判断环节选择 Qwen/Qwen3-VL-4B-Instruct，是因为该环节需要同



时理解关键帧、语音文本、声音摘要和投稿元数据。普通文本大模型只能处理文本化后的信息，难以直接利用关键帧画面；传统图像分类模型虽然能够识别画面类别，但不适合结合文本上下文生成审核理由。Qwen3-VL 系列支持图像、视频与文本等多模态输入，能够进行图文理解和视频相关推理，其中 4B 规模模型在多模态理解能力和推理资源消耗之间具有较好的折中<sup>[36]</sup>。因此，本文选择该模型作为片段级综合审核模型，而不是选择仅支持文本输入的模型或部署成本更高的大规模视觉语言模型。

### 5.7.2 视频预处理与音频提取实现

视频预处理的目标是将原始视频文件转换为后续审核模型可以使用的中间数据。视频预处理与音频提取流程如图 5-17 所示。

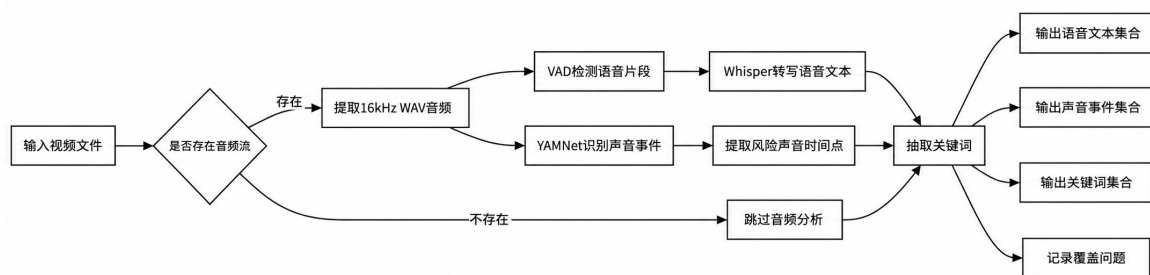


图 5-17 视频预处理与音频提取流程图

系统首先通过 FFmpeg 相关能力检测视频中是否存在音频流，存在则提取出 16kHz 的 WAV 音频文件，供后续 VAD、Whisper 和 YAMNet 使用；不存在则继续进行关键帧抽取和视觉审核，避免纯画面视频因为没有音频而被漏审。

音频提取完成后，系统会形成两条分析路径。第一条路径是语音文本分析，系统先通过 VAD 检测音频中的有效语音片段，再调用 Whisper 将这些语音片段转写为文本。该路径主要用于判断视频中“说了什么”。第二条路径是声音事件分析，系统通过 YAMNet 对音频进行环境声音和异常声音识别。该路径主要用于判断视频中“出现了什么声音”，例如枪声、爆炸声、尖叫声、警报声等。

视频预处理阶段还需要提取关键帧。关键帧是后续 Qwen 多模态审核的重要视觉输入。系统不适合将视频中的所有帧都送入模型，因为这样会造成较大的计算开

销，也会产生大量重复画面。因此按照一定的抽帧频率提取代表帧，并结合风险声音时间点补充附近画面。通过这种方式，可以在控制模型调用次数的同时，提高对短时风险画面的覆盖能力。

视频预处理与音频提取过程如算法 4 所示。

---

**算法 4** 视频预处理与音频提取实现算法

---

**输入：** 视频文件路径 *videoPath*  
**输出：** 语音文本集合 *S*，声音事件集合 *A*，关键帧集合 *F*，覆盖问题集合 *C*

- 1: 初始化  $S \leftarrow \emptyset$ ,  $A \leftarrow \emptyset$ ,  $F \leftarrow \emptyset$ ,  $C \leftarrow \emptyset$
- 2: 检测视频文件 *videoPath* 是否存在音频流
- 3: **if** 存在音频流 **then**
- 4:     使用 FFmpeg 从视频中提取 16kHz WAV 音频文件 *audioPath*
- 5:     **if** 音频提取失败 **then**
- 6:         将音频提取异常写入 *C*
- 7:     **else**
- 8:         使用 VAD 检测语音片段集合 *V*
- 9:         使用 YAMNet 检测声音事件集合 *A*
- 10:        **if** *V* 不为空 **then**
- 11:            调用 Whisper 对语音片段集合 *V* 进行转写，得到语音文本集合 *S*
- 12:        **else**
- 13:            保持  $S \leftarrow \emptyset$ ，后续由视觉关键帧和声音事件继续构建审核窗口
- 14:        **end if**
- 15:     **end if**
- 16: **else**
- 17:     跳过音频分析流程，并记录视频无音频流
- 18: **end if**
- 19: 从声音事件集合 *A* 中提取风险声音时间点集合 *T*
- 20: 根据固定抽帧频率和风险声音时间点集合 *T* 提取关键帧集合 *F*
- 21: **if** *F* 为空 **then**
- 22:     将关键帧提取异常写入 *C*
- 23: **end if**
- 24: 返回 *S*、*A*、*F* 和 *C*

---

通过上述处理，系统将原始视频转换为带有时间戳的语音文本、声音事件和关键帧数据。后续审核流程不再围绕完整视频文件直接展开，而是围绕这些结构化证据构建片段。这样既能降低模型输入规模，也能使审核结果更容易和视频时间轴对应。

### 5.7.3 语音文本、声音事件与关键帧审核实现

完成视频预处理后，系统需要将语音文本、声音事件和关键帧组织为可供 Qwen 审核的时间窗口。以视频画面为主轴构建 *visual\_scene* 场景片段。场景片段的边界

主要来自视频时长、场景切分结果、固定窗口长度以及风险声音附近的补充边界,可以保证即使视频没有语音文本,也能围绕画面和时间轴进入后续审核流程。

在语音文本处理方面,Whisper 的转写结果会被保存为带开始时间和结束时间的文本片段,并尽量保留词级时间戳。visual\_scene 或后续审核窗口内查找对应的语音文本。如果语音片段时间较长,系统会把当前窗口内命中的文字作为主要证据,同时保留整段语音上下文作为辅助信息,避免长语音内容被错误绑定到不相关画面。

在声音事件处理方面,YAMNet 的识别结果会按照时间戳对齐到场景片段和审核窗口中。对于普通声音事件,系统将其作为背景上下文加入提示词;对于枪声、爆炸、尖叫、警报等风险声音,系统会在构建视觉窗口时加入风险声音前后的补充边界,并在后续 adaptive\_review\_window 中标记 risk\_focus 原因。若风险声音附近缺少可用画面,系统会记录覆盖问题,并在综合结论中倾向人工复核。

在视觉处理方面,系统不会将视频中的所有帧直接输入模型,而是先抽取关键帧,并为每个场景片段选择代表帧或生成 contact sheet。随后系统根据配置生成固定窗口 review\_window,或默认生成自适应窗口 adaptive\_review\_window。审核窗口会合并一个或多个 visual\_scene,并携带代表帧、窗口文字、声音摘要、投稿元数据和文本规则预检结果,最后由 Qwen/Qwen3-VL-4B-Instruct 给出片段级判断。

多模态审核片段构建过程如图 5-18 所示。系统先根据视觉场景和风险声音构建场景片段,再按照固定或自适应策略生成 Qwen 审核窗口,并将语音、声音、画面和投稿元数据作为同一时间范围内的上下文组合起来,从而减少重复审核窗口,同时提高无语音视频和低语音视频的覆盖能力。

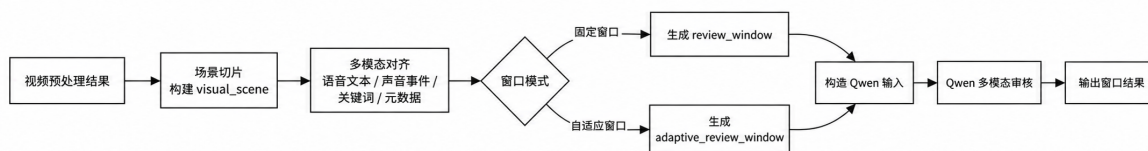


图 5-18 多模态审核片段构建流程图

基于场景窗口的多模态审核片段构建过程如算法 5 所示。该算法的核心思想是:先围绕视频画面和时间轴生成 visual\_scene,再将语音文本、声音事件和投稿元数据对齐到场景片段中,最后生成适合 Qwen 调用的审核窗口。通过这种处理方式,系

统既能覆盖纯画面内容，也能在风险声音或文本规则命中时缩小重点审核范围。

---

**算法 5** 基于场景窗口的多模态审核片段构建算法
 

---

**输入：** 语音文本集合  $T$ ，声音事件集合  $A$ ，关键帧集合  $F$ ，投稿元数据  $M$ ，视频时长  $D$ ，场景切分点集合  $B$

**输出：** Qwen 审核窗口集合  $W$

```

1: 初始化场景片段集合  $S \leftarrow \emptyset$ ，审核窗口集合  $W \leftarrow \emptyset$ 
2: 根据  $D$ 、 $B$ 、固定窗口长度和风险声音时间点生成视觉时间边界
3: 合并过短窗口，得到 visual_scene 时间范围集合
4: for 每一个视觉时间范围  $r$  do
5:   从  $T$  中提取  $r$  内的语音文本和必要的长语音上下文
6:   从  $A$  中提取  $r$  内的普通声音事件和风险声音事件
7:   从  $F$  中选择  $r$  内或附近的代表关键帧
8:   结合  $M$  构建 visual_scene，并记录 modalities 和覆盖问题
9:   将该场景片段加入  $S$ 
10: end for
11: if 审核窗口模式为固定窗口 then
12:   按固定时长从  $S$  中合并场景片段，生成 review_window 并加入  $W$ 
13: else
14:   根据风险声音、文本规则命中和覆盖缺口生成重点窗口
15:   将重点窗口和覆盖窗口合并，生成 adaptive_review_window 并加入  $W$ 
16: end if
17: for 每一个审核窗口  $w \in W$  do
18:   生成代表帧 contact sheet
19:   根据画面、窗口文字、声音摘要、投稿元数据和文本规则预检结果构造 Qwen 输入
20:   调用 Qwen/Qwen3-VL-4B-Instruct 获取审核结果
21:   将风险分数、风险类型、审核建议、判断理由和追踪文件路径写入  $w$ 
22: end for
23: 按窗口开始时间对  $W$  排序
24: return  $W$ 

```

---

算法 5 对应的后台展示效果如图 5-19 和图 5-20 所示。

图 5-19 展示了后台审核详情页中分 P 文件的 AI 审核结果，页面在播放预览区域下方以时间轴方式标记不同审核窗口，并在右侧展示 AI 状态、AI 建议、风险分数、窗口数量和风险来源等信息。图 5-20 展示了某个风险窗口的具体证据，包括时间范围、风险类型、风险分数、模型判断说明、语音文本内容、风险音频标记以及代表关键帧画面。该界面说明系统已将语音文本、声音事件和关键帧按照审核窗口组织为可查看的证据，便于管理员根据片段证据进行人工复核。

构建完成后，每个审核窗口都会保存开始时间、结束时间、来源类型、代表帧路径、语音文本摘要、声音事件摘要、模型审核结果和模型输入追踪文件。窗口来源主要包括 `visual_scene`、`adaptive_review_window` 和 `review_window`。后台管理员查



图 5-19 分 P 文件 AI 审核结果展示界面



图 5-20 风险审核片段证据展示界面

看审核结果时，可以结合 modalities、风险声音标记、文本摘要和代表帧判断风险来源，从而区分该风险主要来自语音内容、声音事件、投稿元数据还是画面证据。

整体来看，该实现方式将语音、声音和画面三类信息按照视觉场景和审核窗口进行组织，使 Qwen 审核输入不再只依赖某一种模态。对于有语音的视频，系统能够结合语音文本与画面进行判断；对于没有语音但存在风险声音的视频，系统能够围绕声音事件补充画面证据；对于无语音或低语音视频，系统仍然可以借助关键帧和视觉窗口进入审核流程，从而提高视频审核的覆盖范围。

#### 5.7.4 综合风险结果生成实现

片段级审核完成后，系统需要将多个 segment 的审核结果汇总为分 P 级结果，再进一步汇总为视频级结果。分 P 级结果用于说明某个视频文件是否存在风险，视频级结果则用于后台审核列表展示和管理员最终审核参考。由于一个视频可能包含多个分 P 文件，因此系统不能只根据某一个片段结果直接决定整个视频的 AI 建议，而是需要按照层级逐步汇总。

综合风险结果生成采用风险优先和人工兜底结合的策略。系统不会简单地因为任一窗口分数较高就直接给出驳回结论，而是综合判断风险类型、风险分数、证据完整性和命中次数。对于直接拒绝类风险、零容忍风险、多个高风险窗口，或高风险时长占比达到阈值的情况，结果会倾向于驳回；对于普通中高风险、模型判断不确定、缺少视觉证据、Qwen 调用不完整或声音事件无法确认的情况，则优先建议人工复核；只有所有审核窗口均未发现明显风险且审核过程完整时，分 P 级结果才建议通过。该策略既能避免高风险内容被低风险窗口稀释，也能降低单个模型判断直接决定发布结果带来的误判风险。

综合风险结果生成流程如图 5-21 所示。

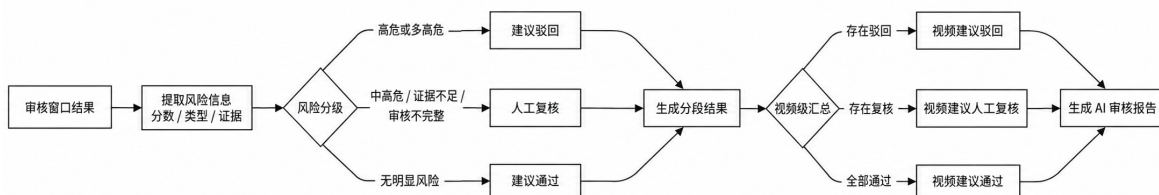


图 5-21 综合风险结果生成流程图

在生成分 P 结果时，系统会遍历该分 P 下的所有审核窗口，提取命中风险的窗口，并记录时间范围、风险类型、风险分数、模型原因、代表帧路径、文本摘要和声音事件摘要。对于证据不足但仍存在风险可能的窗口，系统也会保留覆盖问题和不完整原因，避免后台管理员只能看到一个模糊的“复核”结论。分 P 结果同时包含整体建议和具体证据，便于管理员定位问题内容。

在生成视频级结果时，系统继续对所有分 P 结果进行归并。若任一分 P 建议驳回，则视频级 AI 建议为驳回；若不存在驳回但存在人工复核、处理异常或审核不完整，则视频级 AI 建议为人工复核；只有所有分 P 均建议通过时，视频级 AI 建议才为通过。对于多分 P 视频，视频级结果还会汇总风险分 P、风险窗口数量和主要风险说明，使后台列表能够快速展示风险来源。

最终，AI 服务将视频级结果和分 P 级结果封装为审核结果消息返回主站服务。主站服务接收到结果后，将其写入 AI 审核任务表和分 P 审核明细表，供后台管理端读取。后台管理员查看视频审核详情时，可以根据 AI 审核摘要了解整体风险，也可以根据分 P 明细和片段证据进一步定位具体内容。

通过这种从片段级到分 P 级、再到视频级的逐级汇总方式，系统能够同时满足自动审核和人工复核两方面需求。自动审核给出初步风险判断，分 P 明细保留模型判断依据，人工终审再结合平台规则和视频内容作出最终处理。这样既避免了单纯依赖模型结论带来的误判问题，也提高了管理员定位风险内容的效率。



第 6 章 系统测试与结果分析

本章围绕测试环境、测试方案、核心功能测试、视频处理测试、AI 审核测试、异常场景测试以及测试结果分析展开。为了避免与前文的需求分析、总体设计和详细实现章节重复，本章不再展开说明各模块的内部实现原理，而是重点呈现测试场景、测试步骤、预期结果、实际表现和测试结论。

6.1 测试环境与测试方案

6.1.1 测试环境配置

系统测试环境包括前端运行环境、Java 后端运行环境、Python AI 服务运行环境，以及数据库、中间件、分布式事务组件和视频处理工具等基础组件。测试前需要保证 MySQL、Redis、Elasticsearch、RabbitMQ、Nacos、Seata、FFmpeg 等组件能够正常启动，并依次启动网关服务、主站服务、后台管理服务、资源服务、互动服务和 AI 审核服务。

表 6-1 给出了本系统测试环境的主要配置。

表 6-1 系统测试环境配置

| 环境类别    | 配置项             | 说明                                |
|---------|-----------------|-----------------------------------|
| 操作系统    | Windows / Linux | 用于本地开发、联调和功能演示                    |
| 后端环境    | JDK 17、Maven    | 运行 Spring Boot 与 Spring Cloud 微服务 |
| 前端环境    | Node.js、Vue     | 运行用户端和后台管理端页面                     |
| AI 服务环境 | Python、FastAPI  | 提供音频、图像和视频帧审核接口                   |
| 数据库     | MySQL 8         | 保存用户、视频、投稿、互动和审核数据                |
| 缓存组件    | Redis           | 保存登录态、验证码、上传状态和部分队列数据             |
| 搜索引擎    | Elasticsearch   | 支持视频标题、简介和标签检索                    |
| 消息队列    | RabbitMQ        | 传递 AI 审核任务和审核结果                   |
| 注册中心    | Nacos           | 管理微服务注册发现和配置                      |
| 视频工具    | FFmpeg          | 完成视频转码、切片和音频提取                    |

由表 6-1 可以看出，系统运行依赖多个基础组件。若 RabbitMQ 未启动，则 AI



审核任务无法正常投递；若 Seata 未正常连接，则关键跨服务写操作可能无法完成全局事务协调；若 FFmpeg 路径配置错误，则视频转码流程无法完成；若 Elasticsearch 异常，则搜索功能无法返回正常结果。因此，在正式测试前需要先完成基础环境连通性检查。

6.1.2 测试方案设计

本系统采用功能测试、流程测试和异常测试相结合的方式进行验证。功能测试用于检查单个模块是否满足需求，例如登录、视频播放、评论、弹幕、分类管理等；流程测试用于检查跨模块业务链路是否顺畅，例如视频投稿后能否完成分片上传、转码、AI 审核、人工审核和前台发布；异常测试用于检查系统在未登录访问、权限不足、文件格式错误、服务不可用等情况下是否能够给出合理处理。

表 6-2 展示了系统测试方案。

表 6-2 系统测试方案

| 测试类型    | 测试内容                           | 测试目标          |
|---------|--------------------------------|---------------|
| 用户端功能测试 | 首页浏览、视频搜索、视频播放、评论、弹幕、点赞、收藏、投币  | 验证普通用户功能可正常使用 |
| 创作中心测试  | 封面上传、视频分片上传、投稿提交、稿件状态查询        | 验证创作者投稿流程完整   |
| 后台管理测试  | 管理员登录、用户管理、分类管理、视频审核、AI 审核结果查看 | 验证后台管理功能可用    |
| 视频处理测试  | 分片保存、文件合并、视频转码、HLS 文件生成、播放统计   | 验证资源处理链路正确    |
| AI 审核测试 | 审核任务创建、消息发送、AI 服务处理、结果回写、后台展示  | 验证 AI 审核闭环可用  |
| 异常场景测试  | 未登录操作、权限不足、重复提交、转码失败、AI 服务异常   | 验证系统异常处理能力    |

本章测试重点放在核心业务链路上。对于登录注册、分类管理等常规功能，主要采用接口返回结果和页面显示结果进行验证；对于视频上传、转码、审核和发布等关键流程，则结合数据库状态、文件生成情况、消息队列消费情况和页面展示结果进行综合判断。

## 6.2 系统功能测试

系统功能测试主要验证用户端、创作中心和后台管理端的基础功能是否能够正常使用。系统中的部分功能需要多个接口配合完成，按照用户实际使用过程设计测试用例。

系统功能测试用例如表 6-3 所示。

表 6-3 系统功能测试用例

| 测试内容        | 测试数据                                                                          | 预期结果                                              | 实际结果                                                  |
|-------------|-------------------------------------------------------------------------------|---------------------------------------------------|-------------------------------------------------------|
| 首页视频浏览与分类筛选 | 测试账号：游客；视频：uVCqA6teRT（测试数据 1）；分类：pCategoryId=4。                               | 系统返回对应分类下的视频分页数据，首页能够展示视频封面、标题、作者和播放量等信息。         | 接口返回成功，页面正常显示游戏分类下的视频列表，结果中能够展示测试数据 1，分类筛选结果与数据库记录一致。 |
| 视频搜索与详情加载   | 测试账号：游客；关键词：低头人生；视频：vuVLXyezPv；分 P：EH8J8XibY4KQJtkQ2wc2、ElvGseMcM99fhPjD1cyZ。 | 系统能够返回与关键词相关的视频；进入详情页后能够加载视频标题、简介、标签、作者和分 P 列表。   | 搜索结果中显示“低头人生”，详情页信息和两个分 P 文件列表均能正常加载。                 |
| 评论与弹幕功能     | 测试账号：9874085469；视频：vuVLXyezPv；评论：comment_id=29；弹幕：danmu_id=6（24 秒）。           | 评论和弹幕均能保存成功；刷新页面后，评论区显示新增评论，视频播放到对应时间点时展示弹幕。      | 评论和弹幕记录均存在于数据库中，刷新后评论区和播放器弹幕均能显示对应内容。                 |
| 视频互动功能      | 测试账号：9976639498；视频：uVCqA6teRT；点赞/收藏/投币记录：action_id=22/23/25。                  | 用户能够完成点赞、收藏和投币操作；视频互动统计数据更新；再次进入详情页时显示用户已执行的互动状态。 | 点赞、收藏、投币记录均已写入数据库，视频统计数和用户互动状态能够保留。                   |
| 创作中心投稿信息提交  | 测试账号：9976639498；投稿：XSSgmZJFWr（动物）；分类：pCategoryId=9；状态：status=2。               | 系统能够保存投稿基础信息，并在创作中心列表中展示该投稿记录。                    | 投稿信息已保存到 video_info_post 表，创作中心列表中能够查询到该投稿，状态为待审核。    |
| 后台管理与人工审核   | 管理员账号：admin；投稿：XSSgmZJFWr；AI 任务：task_id=5；人工审核：status=3。                      | 管理员能够查看待审核视频，并对视频执行审核通过操作；审核通过后视频进入用户端展示范围。       | 后台能够查看该投稿及低风险 AI 摘要，人工审核通过后视频状态可流转为审核成功并进入用户端展示范围。    |
| 权限控制测试      | 测试账号：游客和普通用户；测试操作：游客提交评论、普通用户访问后台审核接口、未登录用户提交投稿。                              | 系统应拒绝未登录或无权限操作，并给出登录提示或权限不足提示，不应写入无效业务数据。         | 相关请求被系统拦截，数据库中未产生无效评论、投稿或审核记录。                        |

从表 6-3 可以看出，系统的用户端浏览、搜索、播放、互动、创作中心投稿和后台审核等基础功能均能够按照预期运行。对于公开视频浏览和视频播放等功能，系

统能够支持游客访问；对于评论、弹幕、点赞、收藏、投币和投稿等需要用户身份的功能，系统能够进行登录状态校验；对于后台审核等管理功能，系统能够限制普通用户越权访问。整体来看，系统功能测试结果符合预期。

### 6.3 视频处理与发布链路测试

视频处理与发布链路是本系统较为核心的业务流程, 涉及视频预上传、分片上传、文件合并、转码处理、投稿状态更新、后台审核和用户端播放等多个环节。通过链路测试，可以直观地验证视频从用户投稿到最终发布之间的数据流转是否正确。

视频处理与发布链路测试用例如表 6-4 所示。

表 6-4 视频处理与发布链路测试用例

| 测试内容        | 测试数据                                                                                       | 预期结果                                                       | 实际结果                                                       |
|-------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------|------------------------------------------------------------|
| 正常视频投稿到发布链路 | 测试账号: 9874085469; 视频: BvvusWNd8J (下载 2); 文件: GfXQIPBhffeh1imi8k14; 状态: status=3。           | 系统能够完成封面上传、视频分片上传、文件合并、转码、审核和发布流程; 审核通过后, 用户端可以搜索并播放该视频。   | 视频文件已转码成功, 投稿状态为 status=3, 正式视频表中存在对应记录, 用户端能够正常搜索和播放该视频。  |
| 多分 P 视频处理链路 | 测试账号: 9874085469; 视频: uVCqA6teRT (测试数据 1); 分 P: WFQhOPjrDJXZOfd7QtOt、oUH1OnAAGrNADhq7erby。 | 系统能够为同一投稿保存多个分 P 文件记录, 并分别生成播放资源; 用户端播放页能够展示分 P 列表并支持切换播放。 | 数据库中生成 1 条投稿主记录 and 2 条分 P 文件记录, 两个分 P 均转码成功, 播放页能够正常切换播放。 |
| 短视频处理链路     | 测试账号: 9976639498; 视频: Yr5XeOni5l (打瓦); 文件: y9SYepiMqsdfHHMhEV21; 时长: 120 秒。                | 短视频能够正常完成上传、合并和转码, 生成的播放资源完整, 播放过程中不出现资源缺失。                | 视频文件转码成功, 分 P 文件记录和投稿记录均可查询, 播放器能够加载完整播放资源。                |
| 重复分片上传处理    | 上传标识: 4XE9BgPlq0YX5Vj; 视频: uVCqA6teRT; 分 P: oUH1OnAAGrNADhq7erby; 重复上传第 2 个分片。             | 系统能够处理重复上传的分片, 不应导致文件合并失败, 也不应影响后续转码和投稿状态。                 | 重复分片上传后系统未出现异常, 视频仍能完成合并、转码和发布。                            |
| 审核通过后发布验证   | 视频: vuVLXyezPv (低头人生); 状态: status=3; 搜索关键词: 低头人生。                                          | 审核通过后, 投稿状态更新为审核通过, 用户端首页、搜索页和播放页均可以访问该视频。                 | 视频状态更新为审核通过, 用户端能够搜索到该视频, 并能够正常进入播放页。                      |
| 高风险待复核展示限制  | 视频: Yr5XeOni5l; 投稿状态: status=2; AI 任务: task_id=7; 风险: 暴力 @25.98-37.02s。                    | AI 高风险视频应保留在待人工复核链路中, 人工审核前不应进入用户端公开视频列表。                  | 后台能够展示高风险摘要并提示人工复核, 该视频未写入正式 video_info 发布表。               |

从表 6-4 可以看出，系统能够完成正常视频、多分 P 视频和短视频的上传、转码、审核和播放流程。对于损坏视频、非视频格式文件和 AI 高风险待复核等异常情况，系统不会将相关视频展示到用户端，从而避免无效或不合规内容进入公开视频链路。整体来看，视频处理与发布链路能够满足本系统的基本业务要求。

## 6.4 AI 审核与异常场景测试

AI 审核是本系统区别于普通视频分享平台的重要功能之一。该功能主要在视频转码完成后触发，由系统创建审核任务并通过消息队列将任务发送给 AI 服务进行处理。AI 服务完成处理后，将审核结果回写到后端系统。本节测试 AI 审核链路是否能够正常流转，以及在异常情况下系统是否仍然保留人工审核能力。

AI 审核与异常场景测试用例如表 6-5 所示。

表 6-5 AI 审核与异常场景测试用例

| 测试内容         | 测试数据                                                                             | 预期结果                                                        | 实际结果                                                               |
|--------------|----------------------------------------------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------|
| 正常视频 AI 初审链路 | 视频: XSSgmZJFWr(动物); 分 P: w7oDso9Aa6ombbXmIjHK; AI 任务: task_id=5; 建议: 通过, 低风险。    | 视频转码完成后自动创建 AI 审核任务; AI 服务消费任务并返回低风险结果; 后台审核页面能够显示 AI 审核摘要。 | AI 审核任务创建成功, AI 服务返回 videoDecision=pass 和“低风险 - 建议通过”, 后台能够查看审核摘要。 |
| 风险视频 AI 初审链路 | 视频: Yr5XeOni5l(打瓦); 分 P: y9SYepiMqsdfHHMhEV21; AI 任务: task_id=7; 风险: 暴力, 0.85。   | AI 服务能够识别疑似风险内容, 并返回风险等级、风险原因和审核建议, 后台管理员能够看到风险提示。          | AI 审核结果显示 videoDecision=reject, 命中 2 个高风险片段, 后台页面显示暴力风险摘要并提示人工复核。  |
| 空文本片段视觉兜底审核  | 视频: taQehKU5du; 分 P: wYn150nTnkyB7z6wjDS7; 片段: 0.00-6.84 秒; 来源: visual_fallback。 | 系统不应因为当前片段没有语音文本而跳过审核, 应基于关键帧或画面内容生成审核片段和审核结果。              | 空文本片段仍然生成审核结果, 片段风险类型为 normal, 风险分为 0.25, 后台能够看到基于画面内容形成的审核摘要。     |
| 含声音事件视频审核    | 视频: taQehKU5du; AI 任务: task_id=11; 片段: 66.84-96.84 秒; 风险类型: audio_event。         | 系统能够将声音事件作为辅助审核信息, 生成对应审核片段和结果摘要。                           | 审核结果中包含声音事件提示, 视频级建议为人工复核, 后台显示需要管理员关注的时间范围。                       |
| 多分 P 视频审核汇总  | 投稿: qQDkiVveIa(合集 01); AI 任务: task_id=6; P1/P2 文件均为低风险。                          | 系统能够分别保存每个分 P 的审核结果, 并在视频级审核结果中汇总整体风险。                      | P1 和 P2 均显示低风险并建议通过, 视频级审核摘要汇总两个分 P 的低风险结果。                        |
| AI 审核结果回写    | 请求: 视频: taQehKU5du; 建议: 人工复核; 风险等级: 中风险。                                         | 系统能够将 AI 审核建议、风险等级和摘要写入数据库, 并在后台审核页面中展示。                    | 数据库中审核结果字段更新成功, 后台页面能够显示 AI 审核摘要和风险等级。                             |

从表 6-5 可以看出，AI 审核任务能够在视频转码完成后被创建，并通过消息队列交给 AI 服务处理。AI 服务返回结果后，系统能够将审核摘要、风险等级和审核建议写入数据库，并展示给后台管理员。对于空文本视觉兜底片段、多分 P 视频和含声音事件的视频，系统也能够生成相应审核结果，说明 AI 审核流程具有一定的完整性。

同时，异常场景测试结果表明，当 AI 服务暂时不可用或审核消息未及时消费时，系统仍然能够保留人工审核入口，不会使整个视频审核流程完全中断。对于未登录访问、越权访问、AI 高风险待复核等情况，系统也能够进行基本限制和状态控制，避免异常数据直接进入公开视频发布链路。整体来看，AI 审核功能能够作为人工审核的辅助依据，能够提高审核流程的可观察性和处理效率。

## 6.5 跨服务事务一致性测试

跨服务事务测试主要验证系统在远程调用失败、数据库写入异常或业务校验失败时，是否能够保持关键业务数据状态一致。由于系统引入 Seata 分布式事务组件，测试时重点关注互动计数、用户资产变更、投稿审核状态更新等涉及多个服务协作的写操作<sup>[19]</sup>。

跨服务事务一致性测试用例如表 6-6 所示。

表 6-6 跨服务事务一致性测试用例

| 测试项    | 测试操作               | 实际结果              | 结论 |
|--------|--------------------|-------------------|----|
| 投币事务   | 模拟投币过程中用户资产更新异常    | 互动记录和用户资产未出现单边提交  | 通过 |
| 审核事务   | 模拟审核发布过程中状态迁移失败    | 投稿状态和正式视频数据保持一致   | 通过 |
| 远程调用失败 | 临时关闭被调用服务后执行跨服务写操作 | 系统返回异常，相关数据未被错误提交 | 通过 |

测试结果表明，系统在涉及多个服务的关键写操作中能够通过全局事务或异常回滚机制降低数据不一致风险。对于播放计数、搜索索引更新等非强一致场景，系统仍采用异步补偿方式进行处理。

6.6 测试结果分析

6.6.1 测试结果汇总

综合上述测试结果，系统的主要功能模块均能够按照预期运行。表 6-7 对测试结果进行汇总。

表 6-7 系统测试结果汇总

| 测试模块  | 测试内容                  | 实际结果               | 结论 |
|-------|-----------------------|--------------------|----|
| 用户端功能 | 浏览、搜索、播放、评论、弹幕和互动     | 页面展示正常，交互结果能够保存和更新 | 通过 |
| 创作中心  | 封面上传、分片上传、投稿提交和状态查看   | 投稿流程完整，状态流转正确      | 通过 |
| 后台管理  | 分类管理、用户管理、视频审核和权限控制   | 管理功能可用，普通用户无法越权访问  | 通过 |
| 视频处理  | 分片合并、视频转码、HLS 文件生成和播放 | 视频能够转码并在前台播放       | 通过 |
| AI 审核 | 任务创建、消息投递、结果回写和后台展示   | AI 审核结果能够辅助人工审核    | 通过 |
| 异常处理  | 未登录、权限不足、文件异常和服务异常    | 系统能够拦截或记录异常状态      | 通过 |

由表 6-7 可以看出，系统基本完成了需求分析中提出的核心功能。用户端能够支持视频浏览和互动，创作中心能够完成投稿流程，后台管理端能够完成内容审核，资源服务能够完成视频转码和播放资源生成，AI 审核服务能够为管理员提供审核辅助信息。

## 结 论

系统围绕在线视频分享平台的业务需求，设计并实现了一个基于微服务架构的视频分享系统。系统主要包括用户端、创作中心、后台管理端、资源服务和 AI 审核服务等部分，能够完成视频浏览、搜索、播放、投稿、分片上传、视频转码、用户互动、后台审核和 AI 辅助审核等功能。通过系统测试可以看出，系统主要功能能够正常运行，视频从上传、处理、审核到发布和播放的基本流程能够形成闭环，基本达到了毕业设计的预期目标。

基于技术的特点主要体现在三个方面：一是采用前后端分离和微服务架构，将不同业务模块进行拆分，便于扩展和使用；二是结合分片上传、FFmpeg 转码和 HLS 播放资源生成，实现了视频资源从原始文件到网页播放资源的处理流程；三是借助 RabbitMQ 对视频处理和 AI 审核等耗时任务进行异步处理，提高了业务流程的合理性。

系统的创新点主要体现在 AI 内容审核方面。系统将视频投稿发布流程与多模态 AI 审核相结合，在人工审核之前，对语音文本、声音事件和关键帧画面进行分析，并生成风险等级、风险标签和审核建议。相比单纯依赖人工审核或文本关键词审核的方式，此设计能够在一定程度上提高审核效率，适应更多视频含有风险内容等场景。

系统也有不足之处。系统目前主要在本地环境和小规模数据下完成测试，缺少高并发上传、播放和互动场景下的压力测试。视频资源的存储当前主要采用本地文件目录，后续可以引入对象存储或分布式文件系统进行改进。AI 审核方面，AI 审核的结果仍可能受到模型能力、音视频质量和审核规则影响，存在误判或漏判的可能，后续可以结合人工审核反馈不断完善审核规则和风险标签体系。

总体来看，本文围绕在线视频分享平台的核心业务流程，完成了系统需求分析、架构设计、功能实现和测试验证等工作。系统能够支撑视频从投稿、处理、审核到发布播放的完整流程，并在资源处理、异步任务调度和多模态 AI 辅助审核等方面进行综合实践。虽然系统在性能测试、资源存储和 AI 审核准确性方面仍有进一步优化空间，但整体上已经实现了毕业设计预期目标，也为后续扩展为更完善的视频内容平台提供了一定基础。

## 参考文献

- [1] Lewis J, Fowler M. Microservices: a Definition of This New Architectural Term[Z]. <https://martinfowler.com/articles/microservices.html>. Accessed: 2026-05-03. 2014.
- [2] Newman S. Building Microservices: Designing Fine-Grained Systems[M]. 2nd ed. Sebastopol: O'Reilly Media, 2021.
- [3] Richardson C. Microservices Patterns: With Examples in Java[M]. Shelter Island: Manning Publications, 2018.
- [4] Radford A, Kim J W, Xu T, et al. Robust Speech Recognition via Large-Scale Weak Supervision[C] //Proceedings of Machine Learning Research: Proceedings of the 40th International Conference on Machine Learning: vol. 202. PMLR, 2023: 28492-28518.
- [5] Gemmeke J F, Ellis D P W, Freedman D, et al. Audio Set: An Ontology and Human-Labeled Dataset for Audio Events[C]//2017 IEEE International Conference on Acoustics, Speech and Signal Processing. IEEE, 2017: 776-780.
- [6] Bai J, Bai S, Yang S, et al. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond[Z]. <https://arxiv.org/abs/2308.12966>. arXiv:2308.12966. 2023.
- [7] FFmpeg Developers. FFmpeg Documentation[Z]. <https://ffmpeg.org/documentation.html>. Accessed: 2026-05-03. 2026.
- [8] Pantos R, May W. RFC 8216: HTTP Live Streaming[R]. 8216. RFC Editor, 2017.
- [9] VMware Tanzu. Spring Cloud Reference Documentation[Z]. <https://docs.spring.io/spring-cloud/docs/current/reference/html/>. Accessed: 2026-05-03. 2026.
- [10] Nacos. What is Nacos[Z]. <https://nacos.io/en-us/docs/what-is-nacos.html>. Accessed: 2026-05-03. 2026.
- [11] Redis Ltd. Redis Documentation[Z]. <https://redis.io/docs/latest/>. Accessed: 2026-05-03. 2026.
- [12] Elastic. Elasticsearch Reference[Z]. <https://www.elastic.co/docs/reference/elasticsearch>. Accessed: 2026-05-03. 2026.
- [13] Broadcom. RabbitMQ Documentation[Z]. <https://www.rabbitmq.com/docs>. Accessed: 2026-05-03. 2026.
- [14] TensorFlow. Sound Classification with YAMNet[Z]. <https://www.tensorflow.org/hub/tutorials/yamnet>. Accessed: 2026-05-03. 2024.
- [15] Ramírez S. FastAPI Documentation[Z]. <https://fastapi.tiangolo.com/>. Accessed: 2026-05-03. 2026.
- [16] VMware Tanzu. Spring Boot Reference Documentation[Z]. <https://docs.spring.io/spring-boot/index.html>. Accessed: 2026-05-03. 2026.
- [17] VMware Tanzu. Spring Cloud Gateway Reference Documentation[Z]. <https://docs.spring.io/spring-cloud-gateway/reference/index.html>. Accessed: 2026-05-03. 2026.
- [18] VMware Tanzu. Spring Cloud OpenFeign[Z]. <https://spring.io/projects/spring-cloud-openfeign>. Accessed: 2026-05-03. 2026.
- [19] Apache Software Foundation. What Is Seata?[Z]. <https://seata.apache.org/docs/overview/what-is-seata/>. Accessed: 2026-05-03. 2026.
- [20] Oracle. MySQL 8.0 Reference Manual[Z]. <https://dev.mysql.com/doc/refman/8.0/en/>. Accessed:



- 2026-05-03. 2026.
- [21] MyBatis. MyBatis 3 Reference Documentation[Z]. <https://mybatis.org/mybatis-3/>. Accessed: 2026-05-03. 2026.
  - [22] WHATWG. HTML Living Standard: The Video Element[Z]. <https://html.spec.whatwg.org/multipage/media.html>. Accessed: 2026-05-03. 2026.
  - [23] Vue.js Team. Vue.js Guide[Z]. <https://vuejs.org/guide/introduction>. Accessed: 2026-05-03. 2026.
  - [24] Vite Team. Vite Guide[Z]. <https://vite.dev/guide/>. Accessed: 2026-05-03. 2026.
  - [25] Vue Router Team. Vue Router Documentation[Z]. <https://router.vuejs.org/>. Accessed: 2026-05-03. 2026.
  - [26] Pinia Team. Pinia Documentation[Z]. <https://pinia.vuejs.org/>. Accessed: 2026-05-03. 2026.
  - [27] Element Plus Team. Element Plus Documentation[Z]. <https://element-plus.org/>. Accessed: 2026-05-03. 2026.
  - [28] Axios Contributors. Axios Documentation[Z]. <https://axios-http.com/docs/intro>. Accessed: 2026-05-03. 2026.
  - [29] Kleppmann M. Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems[M]. Sebastopol: O'Reilly Media, 2017.
  - [30] Gorwa R, Binns R, Katzenbach C. Algorithmic Content Moderation: Technical and Political Challenges in the Automation of Platform Governance[J]. Big Data & Society, 2020, 7(1).
  - [31] VMware Tanzu. Spring Security Reference Documentation[Z]. <https://docs.spring.io/spring-security/reference/index.html>. Accessed: 2026-05-03. 2026.
  - [32] Amershi S, Weld D, Vorvoreanu M, et al. Guidelines for Human-AI Interaction[C]//Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems. ACM, 2019: 1-13.
  - [33] ISO/IEC. ISO/IEC 25010:2011 Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models[S]. Geneva, 2011.
  - [34] Maity K, Poornash A S, Saha S, et al. ToxVidLM: A Multimodal Framework for Toxicity Detection in Code-Mixed Videos[C]//Findings of the Association for Computational Linguistics: ACL 2024. 2024: 11130-11142.
  - [35] TensorFlow Hub. Sound Classification with YAMNet[Z]. <https://www.tensorflow.org/hub/tutorials/yamnet>. 2024.
  - [36] Qwen Team. Qwen/Qwen3-VL-4B-Instruct[Z]. <https://huggingface.co/Qwen/Qwen3-VL-4B-Instruct>. 2025.

## 致 谢

值此论文完成之际，首先向我的导师……

致谢正文样式与文章正文相同：宋体、小四；行距：22 磅；间距段前段后均为 0 行。阅后删除此段。